
Applied Reinforcement Learning

Exploration vs Exploitation

Riccardo Berta

2026.09.25

Contents

1	Exploration vs Exploitation	1
1.1	Multi-armed bandit problem	2
1.2	Estimating Action Values	5
1.2.1	Sample-average estimation	5
1.2.2	Incremental estimation	5
1.2.3	Exponential Recency-Weighted Average	7
1.2.4	Variable Step-Size Parameter	12
1.2.5	Robbins–Monro conditions	14
1.3	Basic exploration strategies	18
1.3.1	Pure exploration and pure exploitation	20
1.3.2	Epsilon-Greedy	22
1.3.3	Comparing Exploration Strategies	25
1.3.4	Decaying epsilon-greedy	29
1.3.5	Optimistic Initial Values	33
1.4	Strategic Exploration	36
1.4.1	Softmax method	37
1.4.2	Upper Confidence Bound method	41
1.4.3	Thompson sampling	44
1.4.4	Comparison	48
1.5	Regret: Measuring the Cost of Exploration	49
1.6	Parameter study	53
1.7	From Immediate Rewards to Long-Term Value	56

1 Exploration vs Exploitation

For an agent, **exploration** becomes essential when interacting with an **uncertain environment**, where the MDP is not fully known and therefore cannot simply be used for planning. In this setting, the agent must **learn from experience**, repeatedly deciding which actions to take while still being uncertain about their consequences. This creates a fundamental tension. Almost every decision involves a **trade-off between gathering new information and exploiting the information already available**. Consider a simple everyday example: when we go to our favorite restaurant, should we order our favorite dish once again, knowing that we will probably enjoy it, or should we finally try that other dish we have been curious about? The first choice exploits what we already know, the second explores in the hope of discovering something even better. This is the **exploration–exploitation dilemma**, one of the central challenges of reinforcement

learning. At its core, the problem is deciding **when to acquire new knowledge and when to capitalize on what has already been learned**. The difficulty is that the agent can never be completely sure that the best option it has found so far is actually the best option available. In other words, the question is: **is the good we already know good enough?**

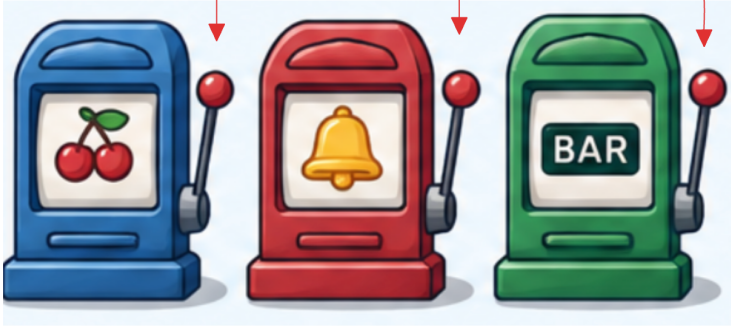
We already encountered this trade-off as one of the three fundamental challenges that make reinforcement learning difficult. The reward provides **evaluative feedback**: it tells the agent how good the action it actually took turned out to be, but it does not reveal what would have happened had the agent chosen one of the other available actions. Those alternatives remain unknown. The only way to learn about them is to **try them**.

To study the exploration–exploitation dilemma in isolation, we temporarily remove the sequential aspect of reinforcement learning and consider environments in which each interaction consists of a single decision. These simplified problems are known as **multi-armed bandits (MABs)**.

1.1 Multi-armed bandit problem

Multi-armed bandits (MABs) are a special case of the reinforcement learning problem in which the state space contains a single state and each interaction has a horizon of one step. A MAB therefore has **multiple actions**, a **single state**, and **no sequential consequences**: each action directly produces a reward, without leading to a new decision-relevant state. We can think of a MAB as a “many-options, single-choice” environment, named by analogy with a slot machine with multiple arms.

A 3-armed bandit is a decision-making problem with two choices



Arm 1
($a = 1$)

Arm 2
($a = 2$)

Arm 3
($a = 3$)

Example of interaction history

t	A_t	R_t	Comment
1	1	0	pulled arm 1
2	2	1	pulled arm 2
3	1	1	pulled arm 1
4	3	0	pulled arm 3
5	1	0	pulled arm 1
...	...	...	...

We need to try them sufficient to correctly assess each options

More formally, at each time step t , the agent chooses one among k possible actions. After selecting the action, it receives a reward sampled from a **stationary probability distribution** associated with that action. The agent does not know these reward distributions and must learn

about them through experience. The true value of an action a is its expected reward:

$$q(a) = \mathbb{E}[R_t \mid A_t = a]$$

Notice that everywhere else in the course the reward produced by acting at time t is written using $t+1$ as a subscript, because the action has to be resolved by the environment before the reward exists. In the bandit setting there is no next state to wait for, so we follow the standard bandit convention and write the reward of the action taken at t using t as a subscript.

The objective of a MAB is closely related to the objective of reinforcement learning, but simpler. In a general RL problem, the agent seeks to maximize the **expected cumulative discounted reward**, because an action can influence not only the immediate reward but also future states and, consequently, future rewards. In a bandit interaction, instead, there are **no future consequences** to consider: an action produces an immediate reward and the interaction ends. We can therefore remove the terms that no longer apply. There is **no cumulative reward** within an interaction because there is only one reward, and there is **no need for discounting** because there are no future rewards to discount. The objective of the agent is simply to **maximize the expected reward**. The word “expected”, however, remains essential. **Rewards are stochastic**: selecting the same action multiple times may produce different rewards. The agent must therefore learn which action has the highest expected reward, rather than simply selecting the action that happened to produce the largest reward in a particular trial.

We can illustrate these ideas with a numerical simulation of a 10-armed bandit problem. When the environment is created, a true action value is randomly assigned to each of the 10 actions. Then, whenever the agent selects an action, the corresponding reward is sampled from a normal distribution centered at that action’s true value, with unit variance:

```
import numpy as np
import matplotlib.pyplot as plt

# Fix the seed to get reproducible results
np.random.seed(42)

# Define a simple multi-armed bandit environment
class BanditEnv:
    def __init__(self, k=10):

        # Define the number of arms
        self.k = k

        # Randomly initialize the mean reward for each arm
        self.means = np.random.randn(self.k)
```

```

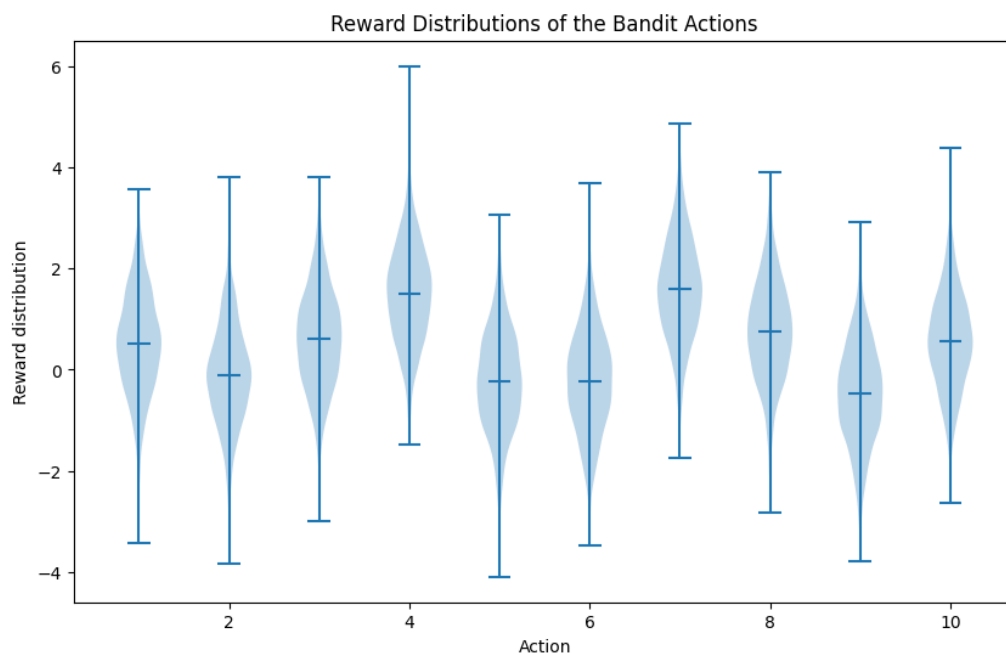
def step(self, action):

    # Return a reward sampled from a normal distribution
    # centered at the arm's mean
    return np.random.normal(loc=self.means[action])

# Create the environment
env = BanditEnv()

# Visualize the reward distributions for each action
plt.figure(figsize=(10,6))
plt.violinplot(np.random.randn(2000, env.k) + env.means, showmeans=True)
plt.xlabel("Action")
plt.ylabel("Reward distribution")
plt.title("Reward Distributions of the Bandit Actions")
plt.show()

```



If we knew the true value of every action, solving the problem would be trivial: we would simply always select the action with the highest value:

$$a_* = \arg \max_a q(a)$$

However, in a bandit problem, we assume that the true action values are unknown. The agent may maintain estimates of these values based on its previous experience, but these estimates

are inherently uncertain.

1.2 Estimating Action Values

Estimating action values in a MAB environment is relatively straightforward. Since each interaction consists of a single action followed by an immediate reward, the value of an action can be **estimated directly** from the rewards observed when that action was selected.

1.2.1 Sample-average estimation

More precisely, we estimate the true action value $q(a)$ **by computing the average** of all rewards obtained from action a up to time t . Thus, the estimated action value is the total reward collected from selecting a , divided by the number of times a has been selected:

$$Q_t(a) = \frac{\sum_{i=1}^{t-1} R_i \mathbb{1}_{[A_i=a]}}{\sum_{i=1}^{t-1} \mathbb{1}_{[A_i=a]}} = \frac{\sum_{i=1}^{t-1} R_i \mathbb{1}_{[A_i=a]}}{N_t(a)}$$

where $\mathbb{1}_{[\cdot]}$ is the indicator function, which equals 1 when its argument is true and 0 otherwise, and $N_t(a)$ is the **number of times action a has been selected before time t** . This counter reappears in every strategy of the chapter: it is the quantity that measures how much evidence we have about an action. This approach is called the **sample-average method**, because the estimated value is simply the average of the reward samples observed for action a . As the action is selected more often, more reward samples become available and the estimate can become increasingly representative of its true expected value.

1.2.2 Incremental estimation

A straightforward implementation of the sample-average method would **store all the rewards observed for each action** and recompute their average whenever the corresponding action-value estimate is needed. However, this approach becomes increasingly inefficient as learning progresses: every new reward requires additional memory, and recomputing the average requires processing an ever-growing number of samples. Fortunately, storing the entire reward history is unnecessary. The sample average can be updated **incrementally**, using only the previous estimate and the newly observed reward. This makes both the memory requirement and the computation per update constant over time. To simplify the notation, from now on we focus on a single action and let the subscript count how many times that particular action has been selected, rather than the global interaction time. Thus, R_n denotes the reward obtained on the

n -th selection of the action, while Q_n denotes its estimated value after the first $n-1$ rewards have been observed. After observing the n -th reward, the new estimate is the average of the first n rewards:

$$Q_{n+1} = \frac{R_1 + R_2 + \dots + R_n}{n} = \frac{1}{n} \sum_{i=1}^n R_i$$

We can separate the most recent reward from the previous ones:

$$Q_{n+1} = \frac{1}{n} \left(R_n + \sum_{i=1}^{n-1} R_i \right) = \frac{1}{n} \left(R_n + (n-1) \frac{1}{n-1} \sum_{i=1}^{n-1} R_i \right)$$

Since the average of the first $n-1$ rewards is precisely Q_n :

$$Q_{n+1} = \frac{1}{n} (R_n + (n-1)Q_n) = \frac{1}{n} (R_n + nQ_n - Q_n)$$

and therefore:

$$Q_{n+1} = Q_n + \frac{1}{n} (R_n - Q_n)$$

We can implement this update rule in a simple Python class that maintains the current estimate of the action value and the number of times the action has been selected:

```
# Define a sample-average estimator for action values
class SampleAverageEstimator:

    def __init__(self, initial_value=0.0):
        # Initialize the estimate
        self.Q = initial_value

        # Initialize the count of updates
        self.n = 0

    # Update the estimate with a new reward
    def update(self, reward):
        # Increment the count of updates
        self.n += 1

        # Update the estimate using the sample-average formula
        self.Q = self.Q + (1 / self.n) * (reward - self.Q)

        # return the updated estimate
        return self.Q

    @property
    def value(self):
```

```
return self.Q
```

This incremental implementation requires storing only the current estimate and the number of times the action has been selected. Each new reward can therefore be processed with constant memory and constant computation. More importantly, this equation introduces an **update pattern** that appears repeatedly throughout reinforcement learning:

$$\text{NewEstimate} \leftarrow \text{OldEstimate} + \text{StepSize} \times (\text{Target} - \text{OldEstimate})$$

The quantity $(\text{Target} - \text{OldEstimate})$ represents an **estimation error**: it measures the discrepancy between the current estimate and a new piece of information indicating where the estimate should move. The step size determines how strongly the estimate reacts to this error. In the sample-average method, the new reward plays the role of the target:

$$Q_{n+1} = Q_n + \frac{1}{\underbrace{n}_{\text{StepSize}}} \left(\underbrace{R_n}_{\text{Target}} - \underbrace{Q_n}_{\text{OldEstimate}} \right)$$

As more rewards are observed, the step size becomes progressively smaller. Consequently, each new reward has a decreasing influence on the estimate, while the accumulated experience from previous rewards becomes increasingly dominant.

1.2.3 Exponential Recency-Weighted Average

The sample-average method is well suited to **stationary bandit problems**, in which the reward distribution associated with each action does not change over time. However, many real-world problems are **non-stationary**: the value of an action may change as the environment evolves. In such cases, observations from the distant past may no longer accurately describe the current action value, so it is desirable to **give more weight to recent rewards** and progressively less weight to older ones. A simple and widely used approach is to replace the decreasing step size with a **constant step-size** parameter:

$$Q_{n+1} = Q_n + \alpha(R_n - Q_n) \quad \alpha \in (0, 1]$$

To understand the effect of using a constant step size, we can expand the update recursively:

$$Q_{n+1} = Q_n + \alpha(R_n - Q_n) = \alpha R_n + (1 - \alpha)Q_n$$

Substituting the previous estimate gives:

$$Q_{n+1} = \alpha R_n + (1 - \alpha) [\alpha R_{n-1} + (1 - \alpha)Q_{n-1}]$$

and therefore:

$$Q_{n+1} = \alpha R_n + \alpha(1 - \alpha)R_{n-1} + (1 - \alpha)^2 Q_{n-1}$$

Continuing recursively:

$$Q_{n+1} = \alpha R_n + \alpha(1 - \alpha)R_{n-1} + \alpha(1 - \alpha)^2 R_{n-2} + \dots + \alpha(1 - \alpha)^{n-1} R_1 + (1 - \alpha)^n Q_1$$

or, more compactly,

$$Q_{n+1} = (1 - \alpha)^n Q_1 + \sum_{i=1}^n \alpha(1 - \alpha)^{n-i} R_i$$

This expression reveals the key property of the method. The weight assigned to rewards decreases exponentially with the age of the observation. **Recent rewards therefore have a greater influence on the current estimate, while older rewards are progressively forgotten.** We can implement this update rule in a simple Python class that maintains the current estimate of the action value and updates it with each new reward:

```
# Define a recency-weighted estimator for action values
class RecencyWeightedEstimator:

    def __init__(self, alpha=0.1, initial_value=0.0):
        # Initialize the step size
        self.alpha = alpha

        # Initialize the estimate
        self.Q = initial_value

    # Update the estimate with a new reward
    def update(self, reward):

        # Update the estimate using the recency-weighted formula
        self.Q = self.Q + self.alpha * (reward - self.Q)

        # return the updated estimate
        return self.Q

    @property
    def value(self):
        return self.Q
```

Notice that all the weights sum to one:

$$(1 - \alpha)^n + \sum_{i=1}^n \alpha(1 - \alpha)^{n-i} = 1$$

This is important because it shows that the update is still a **genuine weighted average**, not just an arbitrary linear combination. Since all coefficients are nonnegative and sum to one, the

estimated value is a **convex combination** of the observed rewards. This guarantees, in particular, that if all those quantities lie in some interval, then the updated estimate also remains in that interval. The proof is just a finite geometric series. Starting from

$$(1 - \alpha)^n + \sum_{i=1}^n \alpha(1 - \alpha)^{n-i}$$

Let $j = n - i$, then:

$$\sum_{i=1}^n \alpha(1 - \alpha)^{n-i} = \alpha \sum_{j=0}^{n-1} (1 - \alpha)^j$$

Using the finite geometric-series formula:

$$\sum_{j=0}^{n-1} r^j = \frac{1 - r^n}{1 - r}$$

with $r = 1 - \alpha$:

$$\alpha \sum_{j=0}^{n-1} (1 - \alpha)^j = \alpha \frac{1 - (1 - \alpha)^n}{1 - (1 - \alpha)} = \alpha \frac{1 - (1 - \alpha)^n}{\alpha} = 1 - (1 - \alpha)^n$$

Therefore:

$$(1 - \alpha)^n + \sum_{i=1}^n \alpha(1 - \alpha)^{n-i} = (1 - \alpha)^n + 1 - (1 - \alpha)^n = 1$$

So the weights form a proper convex combination. Intuitively, as n grows, the weight assigned to the initial estimate decreases, while the remaining weight is progressively redistributed among the observed rewards. Thus, the estimated value is a weighted average of all the observed rewards. Because the influence of past observations decays exponentially with their age, this method is known as **exponential recency-weighted average**. The parameter controls **how quickly the agent forgets the past**: a larger value makes the estimate **respond more rapidly** to recent changes, whereas a smaller value produces a **smoother estimate** that retains information from older observations for longer.

To show the effectiveness of this method, we use a standard non-stationary testbed. All ten arms start with a random true value, and at every step each action value takes an independent **random-walk step**. The optimal arm therefore changes over time, and an agent that stops learning is guaranteed to fall behind:

```
# Define a non-stationary multi-armed bandit environment
class NonStationaryBanditEnv:
    def __init__(self, k=10, drift=0.01):

        # Define the number of arms
        self.k = k
```

```
# Define the drift parameter
self.drift = drift

# Randomly initialize the mean reward for each arm
self.means = np.random.randn(self.k)

def step(self, action):

    # Calculate the reward for the selected action
    reward = np.random.normal(loc=self.means[action])

    # Update the mean rewards for each arm with a small random drift
    self.means = self.means + np.random.normal(scale=self.drift, size=self.k)

    # Return the reward for the selected action
    return reward
```

To see how these two estimators behave in practice, we can track a single action in the non-stationary bandit and compare the sample-average estimate with an exponential recency-weighted estimate using a constant step size:

```
# Create a non-stationary bandit
env = NonStationaryBanditEnv(k=10, drift=0.01)

# Track one arm only
action = 0

# Number of interactions
steps = 5000

# Create the estimators
sample_average = SampleAverageEstimator()
recency_weighted = RecencyWeightedEstimator(alpha=0.05)

# Store trajectories
true_values = []
average_estimates = []
recency_estimates = []

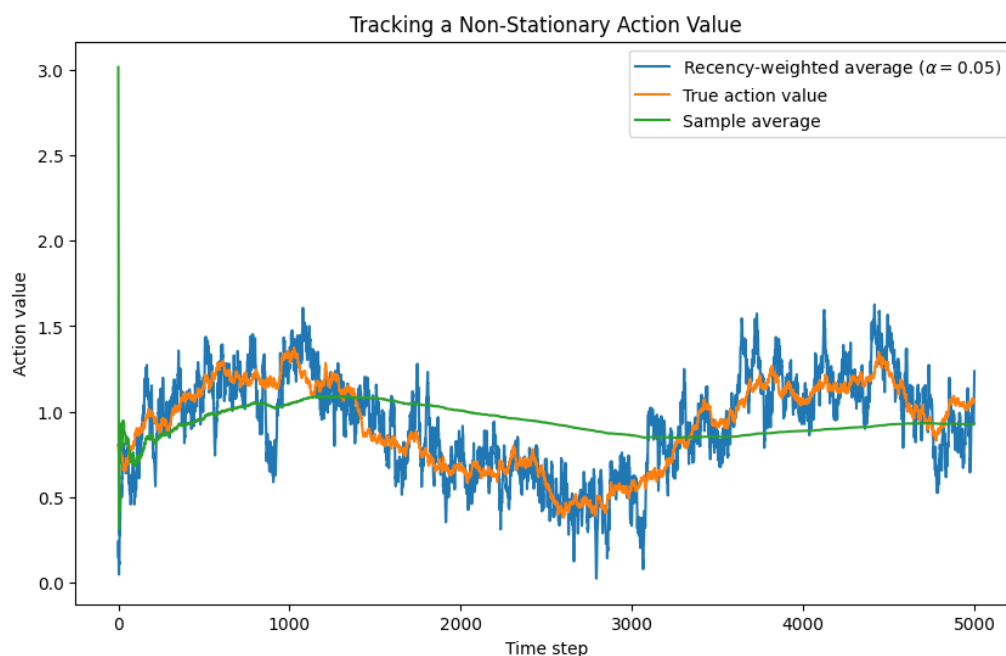
# Simulate the interactions with the environment
for _ in range(steps):
```

```
# Store the true value before the environment drifts
true_values.append(env.means[action])

# Sample reward
reward = env.step(action)

# Update the estimators
average_estimates.append(sample_average.update(reward))
recency_estimates.append(recency_weighted.update(reward))

# Visualize the results
plt.figure(figsize=(10, 6))
plt.plot(recency_estimates, label=fr"Recency-weighted average
↪ ($\alpha={recency_weighted.alpha})")
plt.plot(true_values, label="True action value")
plt.plot(average_estimates, label="Sample average")
plt.xlabel("Time step")
plt.ylabel("Action value")
plt.title("Tracking a Non-Stationary Action Value")
plt.legend()
plt.show()
```



The figure illustrates the expected behavior quite well. The true action value (orange) changes

over time because of the random drift, so the quantity to be estimated is itself a moving target. The sample-average estimate (green) works reasonably well at the beginning, but progressively loses its ability to track the true value. As more samples are collected, its step size approaches zero. Consequently, new rewards have an increasingly small effect on the estimate. The recency-weighted estimate (blue) behaves very differently. Because it uses a constant step, it continues to react to new observations throughout the experiment and therefore follows the changes in the true action value. However, the curve is visibly **noisy**. This is the other side of using a relatively large step size: the estimator reacts not only to changes in the underlying action value, but also to the stochastic noise of individual rewards. The parameter α controls the **trade-off between adaptability and noise suppression**.

1.2.4 Variable Step-Size Parameter

Sometimes it is useful to **vary the step size over time**, rather than using either the sample-average step size or a fixed value. Let α_n denote the step-size parameter used when processing the n -th reward observed for a given action. The update rule then becomes:

$$Q_{n+1} = Q_n + \alpha_n(R_n - Q_n)$$

Expanding the estimate gives:

$$Q_{n+1} = \alpha_n R_n + (1 - \alpha_n)Q_n = \alpha_n R_n + (1 - \alpha_n) [\alpha_{n-1} R_{n-1} + (1 - \alpha_{n-1})Q_{n-1}]$$

and therefore:

$$Q_{n+1} = \alpha_n R_n + (1 - \alpha_n)\alpha_{n-1} R_{n-1} + (1 - \alpha_n)(1 - \alpha_{n-1})Q_{n-1}$$

Continuing recursively:

$$Q_{n+1} = \alpha_n R_n + (1 - \alpha_n)\alpha_{n-1} R_{n-1} + (1 - \alpha_n)(1 - \alpha_{n-1})\alpha_{n-2} R_{n-2} + \dots + (1 - \alpha_n)(1 - \alpha_{n-1}) \dots (1 - \alpha_1)Q_1$$

Using product notation:

$$Q_{n+1} = \left[\prod_{i=1}^n (1 - \alpha_i) \right] Q_1 + \sum_{i=1}^n \left[\alpha_i \prod_{j=i+1}^n (1 - \alpha_j) \right] R_i$$

This expression makes the role of the step-size sequence explicit. Reward R_i enters the estimate with weight α_i , and every subsequent update multiplies its remaining influence by $(1 - \alpha_j)$. Thus, its weight after n updates is:

$$\alpha_i \prod_{j=i+1}^n (1 - \alpha_j)$$

The sequence $\alpha_1, \alpha_2, \dots$ therefore determines how quickly past information is forgotten. For the implementation, we use a generic estimator that accepts any step-size schedule:

```
class VariableStepSizeEstimator:
    def __init__(self, step_size, initial_value=0.0):
        # Store the function that determines the step size at each update
        self.step_size = step_size

        # Initialize the action-value estimate
        self.Q = initial_value

        # Initialize the number of observed rewards
        self.n = 0

    def update(self, reward):

        # Increment the number of observed rewards
        self.n += 1

        # Compute the step size for the current update
        alpha = self.step_size(self.n)

        # Update the action-value estimate
        self.Q = self.Q + alpha * (reward - self.Q)

        # Return the updated estimate
        return self.Q

    @property
    def value(self):
        return self.Q
```

The two estimators introduced earlier are special cases. Setting:

$$\alpha_i = 1/i$$

gives the sample-average method.

```
sample_average = VariableStepSizeEstimator(step_size=lambda n: 1 / n)
```

While setting:

$$\alpha_i = \alpha$$

gives the exponential recency-weighted average.

```
recency_weighted = VariableStepSizeEstimator(step_size=lambda n: 0.1)
```

1.2.5 Robbins–Monro conditions

For a stationary bandit, the sample-average choice converges to the true action value under the usual assumptions, by the law of large numbers. More generally, **convergence is not guaranteed for an arbitrary sequence of step sizes**. Stochastic approximation theory gives two classical conditions, commonly known as the **Robbins–Monro conditions**. The first condition ensures that **the total amount of adaptation remains large enough** to overcome the influence of the initial estimate and continue moving toward the correct value. If the step sizes decrease too quickly, learning may effectively stop before the estimate reaches its target:

$$\sum_{n=1}^{\infty} \alpha_n = \infty$$

To see why this matters, consider the generic update rule toward a fixed target:

$$Q_{n+1} = Q_n + \alpha_n (q - Q_n)$$

Defining the error as

$$e_n = Q_n - q$$

we obtain:

$$e_{n+1} = (1 - \alpha_n) e_n$$

and therefore:

$$e_{n+1} = e_1 \prod_{i=1}^n (1 - \alpha_i)$$

For the initial error to disappear, we need:

$$\prod_{i=1}^n (1 - \alpha_i) \rightarrow 0$$

Using

$$\log(1 - x) \approx -x$$

for small x :

$$\log \left(\prod_{i=1}^n (1 - \alpha_i) \right) = \sum_{i=1}^n \log(1 - \alpha_i) \approx - \sum_{i=1}^n \alpha_i$$

Thus, if the step sizes accumulate without bound, the logarithm tends to negative infinity, and therefore the product tends to zero. Intuitively, the algorithm keeps making enough total progress to eventually forget its initial condition.

The second condition ensures that **the updates eventually become small enough** for the effect of stochastic reward noise to vanish. If the squared step sizes accumulate indefinitely, random fluctuations can continue perturbing the estimate forever:

$$\sum_{n=1}^{\infty} \alpha_n^2 < \infty$$

Suppose that the target is observed with zero-mean noise:

$$R_n = q + \eta_n$$

Then the noise injected at step n is scaled by:

$$\alpha_n \eta_n$$

If the noise terms have finite variance, the variance of this contribution is proportional to:

$$\alpha_n^2 \sigma^2$$

Therefore, requiring:

$$\sum_{n=1}^{\infty} \alpha_n^2 < \infty$$

ensures that the accumulated effect of stochastic noise remains finite.

The sample-average schedule satisfies both conditions:

$$\sum_{n=1}^{\infty} \frac{1}{n} = \infty$$

$$\sum_{n=1}^{\infty} \frac{1}{n^2} < \infty$$

A constant step size instead, violates the second condition:

$$\sum_{n=1}^{\infty} \alpha^2 = \infty$$

Therefore, a constant-step-size estimator does not generally settle to a single value in a stationary stochastic problem. It continues to react to newly observed rewards. In a non-stationary environment, however, **this is precisely the desired behavior**: the estimator retains the ability to follow a moving action value rather than eventually becoming insensitive to new data.

A useful family of variable schedules is:

$$\alpha_n = \frac{1}{n^p}$$

The Robbins–Monro conditions reveal an interesting result. The first condition requires $p \leq 1$, while the second requires $2p > 1$. Thus:

$$\frac{1}{2} < p \leq 1$$

provides a family of step-size schedules satisfying both convergence conditions.

```
robbins_monro = VariableStepSizeEstimator(step_size= $\lambda$  n:  $1 / n^{0.7}$ )
```

A particularly instructive experiment is to compare these three schedules on the same non-stationary action.

```
# Create a non-stationary bandit
env = NonStationaryBanditEnv(k=10, drift=0.01)

# Track one arm only
action = 0

# Number of interactions
steps = 5000

# Store trajectories
true_values = []
sample_estimates = []
rm_estimates = []
recency_estimates = []

# Simulate the interactions
for _ in range(steps):

    # Store the current true action value
    true_values.append(env.means[action])

    # Observe a reward
    reward = env.step(action)

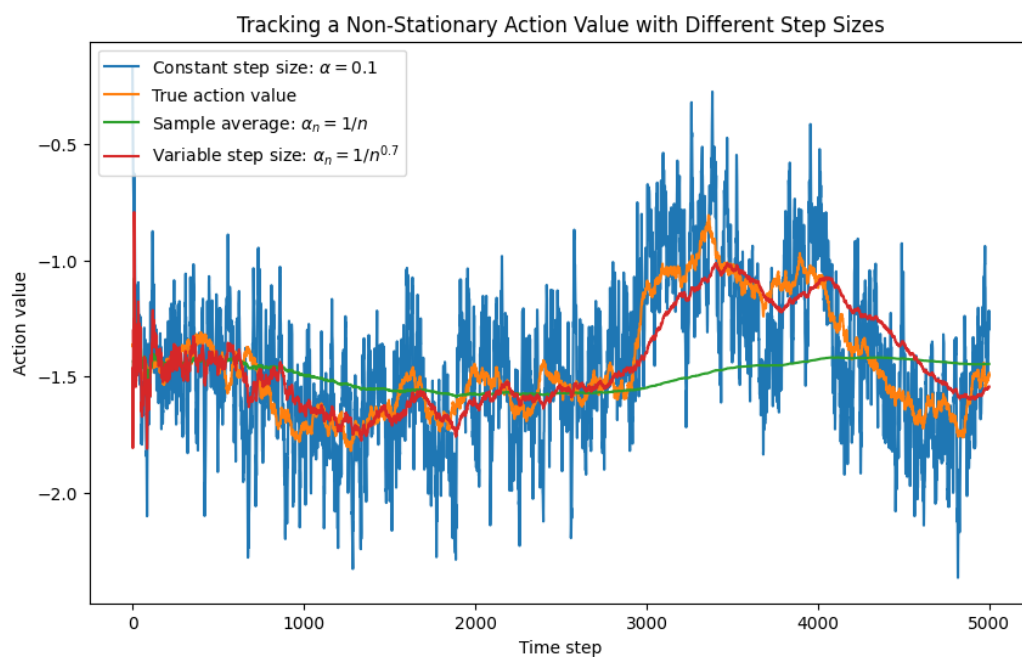
    # Update all estimators using exactly the same reward
    sample_estimates.append(sample_average.update(reward))
    rm_estimates.append(robbins_monro.update(reward))
    recency_estimates.append(recency_weighted.update(reward))

# Visualize the results
plt.figure(figsize=(10, 6))

plt.plot(recency_estimates, label=r"Constant step size:  $\alpha=0.1$ ")
plt.plot(true_values, label="True action value")
plt.plot(sample_estimates, label=r"Sample average:  $\alpha_n=1/n$ ")
```

```
plt.plot(rm_estimates, label=r"Variable step size:  $\alpha_n=1/n^{0.7}$ ")

plt.xlabel("Time step")
plt.ylabel("Action value")
plt.title("Tracking a Non-Stationary Action Value with Different Step Sizes")
plt.legend()
plt.show()
```



This experiment exposes an important distinction that is easy to miss from convergence theory alone. The selected variable step-size schedule satisfies the Robbins–Monro conditions, making it suitable for a stationary problem in which the objective is to converge to a fixed action value. However, because α_n gradually approaches zero, the estimator also becomes progressively less responsive to new observations. Interestingly, compared with the sample-average schedule, the variable schedule decreases more slowly. It therefore retains a greater ability to track changes in the action value, as clearly visible in the figure. The constant step-size estimator represents the other extreme. Since α never decreases, it remains permanently responsive and can continuously track non-stationarity. However, this responsiveness comes at a cost: it also reacts strongly to the stochastic fluctuations of individual rewards, producing a much noisier estimate. The three curves therefore illustrate a fundamental trade-off: decreasing the step size promotes convergence and suppresses noise, while maintaining a larger step size improves the ability to track changes in a non-stationary environment. The variable schedule provides an intermediate behavior: it adapts faster than the sample average, but its tracking ability will

eventually decrease as its step size continues to approach zero.

Finally, it is useful to understand what happens to the **initial estimate**. Before observing any reward, it represents our initial belief about the value of the action. As new rewards are observed, the estimator should progressively replace this prior belief with information coming from actual experience. From the general weighted-average expression, its contribution after n updates is:

$$\left[\prod_{i=1}^n (1 - \alpha_i) \right] Q_1$$

Each update therefore retains only a fraction of the influence that the initial estimate had before that update. In other words, every new observation makes the estimator forget a little more of its initialization. For the sample-average method, the first reward therefore gives zero weight to the initial estimate, and it is completely forgotten after the first observation. From that point onward, the estimate depends only on observed rewards. With a constant step size, instead the initial estimate is not discarded immediately: its influence fades exponentially as experience accumulates. A larger α forgets the initialization more quickly, while a smaller α preserves its influence for longer. This also explains why the choice of a good initial estimate can matter during the **early stages of learning**: before enough experience has been collected, the agent's estimates still partially reflect what we told it to believe at initialization.

1.3 Basic exploration strategies

If we maintain estimates of the action values, then at every time step there is at least one action whose estimated value is maximal. We call such an action a **greedy action**. Selecting a greedy action means **exploiting** our current knowledge: given everything the agent has learned so far, this is the action it currently believes to be best. If several actions share the highest estimated value, one of them can be selected according to some tie-breaking rule, for example uniformly at random.

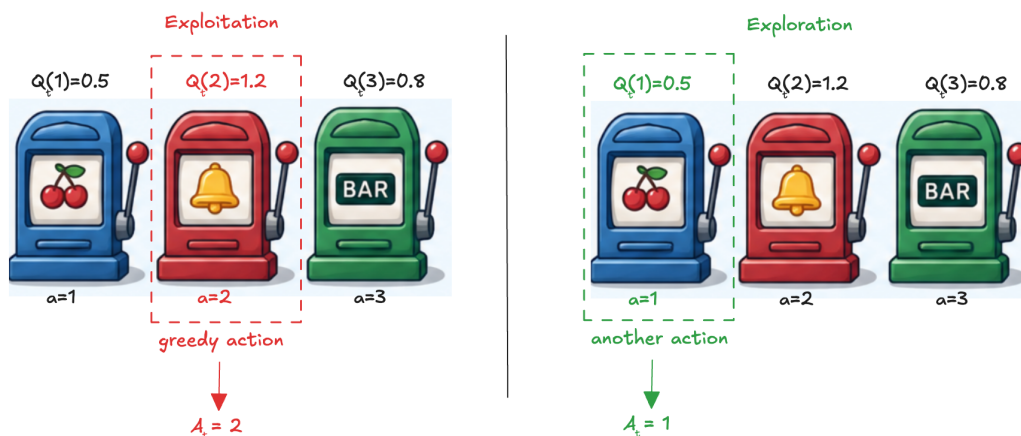
$$A_t \in \arg \max_a Q_t(a)$$

Greedy action selection is **pure exploitation**. It always chooses the action that currently **appears best** and therefore attempts to maximize the expected immediate reward **according to the current estimates**. However, it spends no time trying actions that currently appear inferior. As a consequence, the agent may never discover that one of those actions is actually better than the current greedy action.

When the agent deliberately selects an action other than the greedy one, it is **exploring**. The immediate reward may be lower, but the observation provides new information about that action

and can improve its value estimate. Exploration can therefore **sacrifice reward now in exchange for information** that may lead to better decisions later.

This is the essence of the **exploration–exploitation dilemma**: exploitation maximizes reward according to what the agent currently knows, exploration improves what the agent knows, potentially increasing future reward.



The difficulty is that a single action cannot, in general, simultaneously provide pure exploitation and pure exploration. Choosing the action currently believed to be best uses existing information, whereas trying an uncertain alternative acquires new information. The agent must therefore decide how much immediate reward it is willing to sacrifice in order to make better-informed decisions in the future. This tension is **one of the distinctive challenges of reinforcement learning**. The appropriate balance also depends strongly on the **time horizon**. Suppose the agent knows that it has only one interaction remaining. There is little value in acquiring new information, because there will be no future decision in which that information can be exploited. Acting greedily is therefore the natural choice. If many interactions remain, however, exploration becomes potentially valuable: a reward sacrificed today may reveal a better action that can then be exploited repeatedly in the future. In this sense, the value of information depends on having enough future opportunities to use it. There is also an important **asymmetry** between exploitation and exploration. Exploitation has a relatively clear prescription: choose what currently appears best. Exploration does not. Knowing that more information would be useful does not tell us which action should be explored. We might explore randomly, favor actions that have been tried less often, investigate actions whose values are particularly uncertain, or prefer actions that are sufficiently promising that resolving their uncertainty could change which action we believe to be best. Different exploration strategies embody different answers to the same fundamental question: **which information is worth acquiring, given that acquiring it may cost reward?**

Notice that from now on we use a **vectorized representation** of the action-value estimates. In

the previous section, we focused on estimating the value of a single action, so we encapsulated its estimate and update count in an estimator class. We now need to consider all actions simultaneously, so we store their estimates in an array Q and their selection counts in an array N . This representation will make the action-selection strategies that follow simpler and more natural to implement.

1.3.1 Pure exploration and pure exploitation

The **greedy strategy**, which always exploits, and the pure **random strategy**, which always explores, represent the two **extremes** of the exploration–exploitation trade-off. They are not particularly effective strategies on their own, but they provide two useful **baselines** against which more sophisticated exploration strategies can be compared. To isolate the effect of the action-selection strategy, in the following experiments we use the same sample-average estimator for all strategies. Since the bandit environments considered here are stationary, there is no need to give more weight to recent rewards: all observations remain equally informative about the true action values. The sample-average method is therefore the natural choice and, as each action is sampled repeatedly, its estimate converges toward the corresponding true action value. Using the same estimator throughout also ensures that any differences we observe are due to how actions are selected, rather than to how their values are estimated.

```
def pure_exploitation(env, n_episodes=1000):

    # Initialize the action-value estimates and action-selection counts
    Q = np.zeros(env.k, dtype=float)
    N = np.zeros(env.k, dtype=int)

    # Store data for later analysis
    Q_history = np.empty((n_episodes, env.k), dtype=float)
    rewards = np.empty(n_episodes, dtype=float)
    actions = np.empty(n_episodes, dtype=int)

    # Interact with the environment
    for e in range(n_episodes):

        # Select uniformly at random among the greedy actions
        action = np.random.choice(np.flatnonzero(Q == Q.max()))

        # Execute the selected action and observe the reward
        reward = env.step(action)
```

```
# Update the number of times the selected action has been chosen
N[action] += 1

# Update its estimated value using the sample-average method
Q[action] = Q[action] + (reward - Q[action]) / N[action]

# Store data for later analysis
Q_history[e] = Q
rewards[e] = reward
actions[e] = action

return rewards, Q_history, actions
```

```
def pure_exploration(env, n_episodes=1000):

    # Initialize the action-value estimates and action-selection counts
    Q = np.zeros(env.k, dtype=float)
    N = np.zeros(env.k, dtype=int)

    # Store data for later analysis
    Q_history = np.empty((n_episodes, env.k), dtype=float)
    rewards = np.empty(n_episodes, dtype=float)
    actions = np.empty(n_episodes, dtype=int)

    # Interact with the environment
    for e in range(n_episodes):

        # Select an action uniformly at random, ignoring the current estimates
        action = np.random.randint(env.k)

        # Execute the selected action and observe the reward
        reward = env.step(action)

        # Update the number of times the selected action has been chosen
        N[action] += 1

        # Update its estimated value using the sample-average method
        Q[action] = Q[action] + (reward - Q[action]) / N[action]

        # Store data for later analysis
        Q_history[e] = Q
        rewards[e] = reward
```

```
actions[e] = action

return rewards, Q_history, actions
```

1.3.2 Epsilon-Greedy

We can now combine the two baselines (pure exploitation and pure exploration) into a simple strategy that allows the agent to exploit what it has already learned while continuing to gather information about alternative actions. The idea is straightforward: **act greedily most of the time**, but **occasionally choose an action at random**. This is known as the epsilon-greedy strategy. At each time step, the agent selects a greedy action with probability $(1 - \varepsilon)$ and explores with probability ε by choosing uniformly at random among all k actions. The parameter:

$$\varepsilon \in [0, 1]$$

therefore directly controls the amount of exploration: $\varepsilon = 0$ corresponds to pure exploitation, whereas $\varepsilon = 1$ corresponds to pure random exploration. Notice that the exploratory choice is made among all k actions, including the greedy action itself. Assuming a unique greedy action, it can therefore be selected either during exploitation or by chance during exploration. Its total probability of being selected is:

$$P(A_t = a_{\text{greedy}}) = 1 - \varepsilon + \frac{\varepsilon}{k}$$

while each non-greedy action is selected with probability:

$$P(A_t = a) = \frac{\varepsilon}{k}$$

Thus, ε is **the probability of entering the exploration mode**, not necessarily the probability of selecting a non-greedy action.

```
def epsilon_greedy(env, epsilon=0.01, n_episodes=1000):

    # Initialize the action-value estimates and action-selection counts
    Q = np.zeros(env.k, dtype=float)
    N = np.zeros(env.k, dtype=int)

    # Store data for later analysis
    Q_history = np.empty((n_episodes, env.k), dtype=float)
    rewards = np.empty(n_episodes, dtype=float)
    actions = np.empty(n_episodes, dtype=int)

    # Interact with the environment
```

```
for e in range(n_episodes):

    # Explore with probability epsilon
    if np.random.rand() < epsilon:

        # Select an action uniformly at random
        action = np.random.randint(env.k)

    else:

        # Exploit by selecting uniformly at random among the greedy actions
        action = np.random.choice(np.flatnonzero(Q == Q.max()))

    # Execute the selected action and observe the reward
    reward = env.step(action)

    # Update the number of times the selected action has been chosen
    N[action] += 1

    # Update its estimated value using the sample-average method
    Q[action] = Q[action] + (reward - Q[action]) / N[action]

    # Store data for later analysis
    Q_history[e] = Q
    rewards[e] = reward
    actions[e] = action

return rewards, Q_history, actions
```

We can now verify whether the agent has learned accurate action-value estimates. For each action, we plot its true reward distribution and superimpose the final estimate learned by the epsilon-greedy agent. If learning is successful, each estimated value should lie close to the center of the corresponding reward distribution.

```
# Create a stationary bandit
env = BanditEnv(k=10)

# Run the epsilon-greedy strategy
_, Q_history, _ = epsilon_greedy(
    env=env,
    epsilon=0.5,
    n_episodes=10000
```

```
)

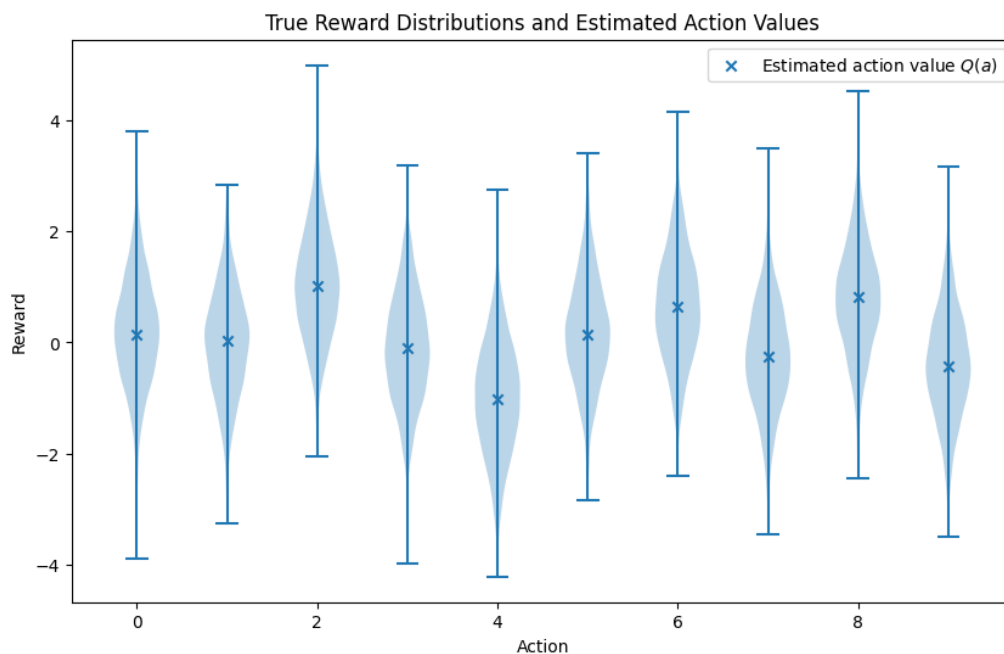
# Sample from the true reward distribution of each action
reward_samples = np.random.randn(2000, env.k) + env.means

# Compare true reward distributions with the learned action values
plt.figure(figsize=(10, 6))

plt.violinplot(reward_samples, positions=range(env.k))

plt.scatter(
    range(env.k),
    Q_history[-1],
    marker="x",
    label="Estimated action value  $Q(a)$ "
)

plt.xlabel("Action")
plt.ylabel("Reward")
plt.title("True Reward Distributions and Estimated Action Values")
plt.legend()
plt.show()
```



1.3.3 Comparing Exploration Strategies

A single run on a single bandit problem tells us very little about the **quality of an exploration strategy**. Both the bandit itself and the rewards it generates are stochastic, so an algorithm may perform particularly well or poorly simply by chance. To obtain a reliable picture of its behavior, we evaluate each strategy **over a number of interaction steps** and repeat the experiment **on several independently generated k-armed bandit problems**. Averaging across these independent runs reveals the typical learning behavior of the strategy rather than the peculiarities of a particular bandit instance. The average reward is the most immediate performance measure, but it does not tell the whole story. We therefore record three complementary quantities at every time step, each averaged over the independent runs:

1. the **average reward**, which measures how much reward the strategy actually obtains;
2. the **optimal-action rate**, defined as the fraction of runs in which the strategy selects the true optimal action at that step, this reveals whether the agent has learned to identify and exploit the best action;
3. the **cumulative regret**, which measures how much expected reward the strategy has lost so far compared with an ideal agent that always selects the optimal action. Unlike the previous two measures, regret explicitly captures the cost of exploration and incorrect decisions accumulated over time.

Since all three quantities can be obtained from the same simulation, we compute them together:

```
# Evaluate a strategy over multiple independent bandit problems
def evaluate(strategy, envs, **kwargs):

    # Store rewards, optimal-action selections, and cumulative regret
    rewards = []
    optimal_actions = []
    cumulative_regrets = []

    # Evaluate the strategy independently on each bandit problem
    for env in envs:

        # Run the strategy and collect rewards and selected actions
        r, _, actions = strategy(env, **kwargs)

        # Identify the true optimal action for this bandit
        best_action = np.argmax(env.means)
```

```
# Store the reward obtained at each step
rewards.append(r)

# Record whether the optimal action was selected at each step
optimal_actions.append(actions == best_action)

# Compute the expected loss at each step relative to the optimal action
regret = env.means[best_action] - env.means[actions]

# Accumulate the regret over time
cumulative_regrets.append(np.cumsum(regret))

# Average the performance measures across all bandit problems
return (
    np.mean(rewards, axis=0),
    np.mean(optimal_actions, axis=0),
    np.mean(cumulative_regrets, axis=0)
)
```

We then generate the independent bandit problems and evaluate all strategies on the same set of problems, making their comparison fair:

```
# Number of independent bandit problems
repetitions = 2000

# Generate the bandit problems used by all strategies
envs = [BanditEnv() for _ in range(repetitions)]

# Evaluate the different action-selection strategies
res_exploitation = evaluate(pure_exploitation, envs)
res_epsilon_02 = evaluate(epsilon_greedy, envs, epsilon=0.2)
res_epsilon_001 = evaluate(epsilon_greedy, envs, epsilon=0.01)
res_exploration = evaluate(pure_exploration, envs)
```

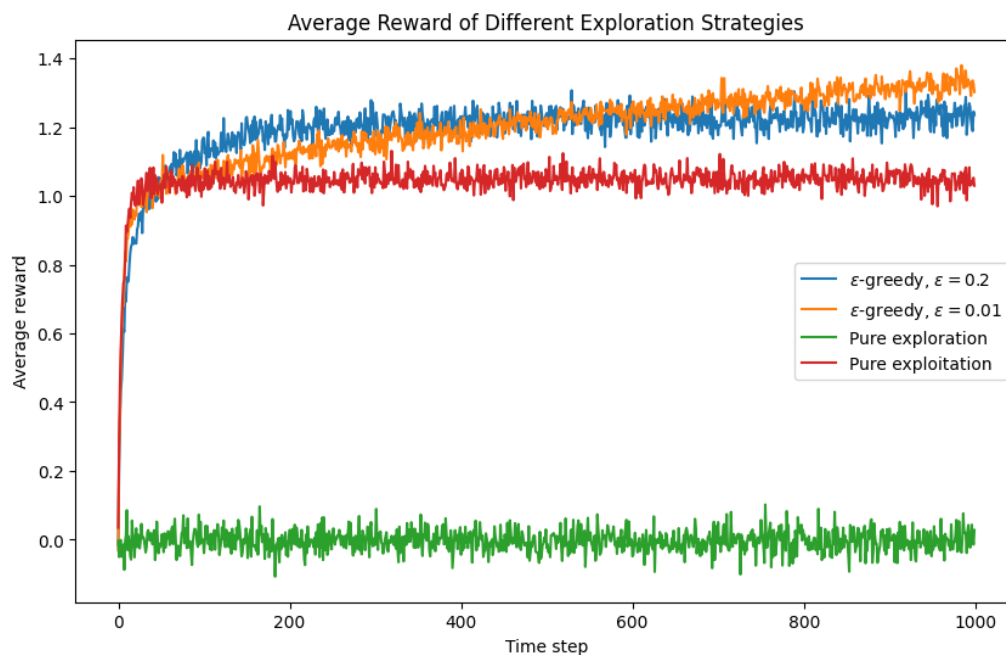
We can first compare the strategies in terms of the average reward obtained at each interaction step:

```
plt.figure(figsize=(10, 6))

plt.plot(res_epsilon_02[0], label=r"$\epsilon$-greedy, $\epsilon=0.2$")
plt.plot(res_epsilon_001[0], label=r"$\epsilon$-greedy, $\epsilon=0.01$")
```

```
plt.plot(res_exploration[0], label="Pure exploration")
plt.plot(res_exploitation[0], label="Pure exploitation")

plt.xlabel("Time step")
plt.ylabel("Average reward")
plt.title("Average Reward of Different Exploration Strategies")
plt.legend()
plt.show()
```



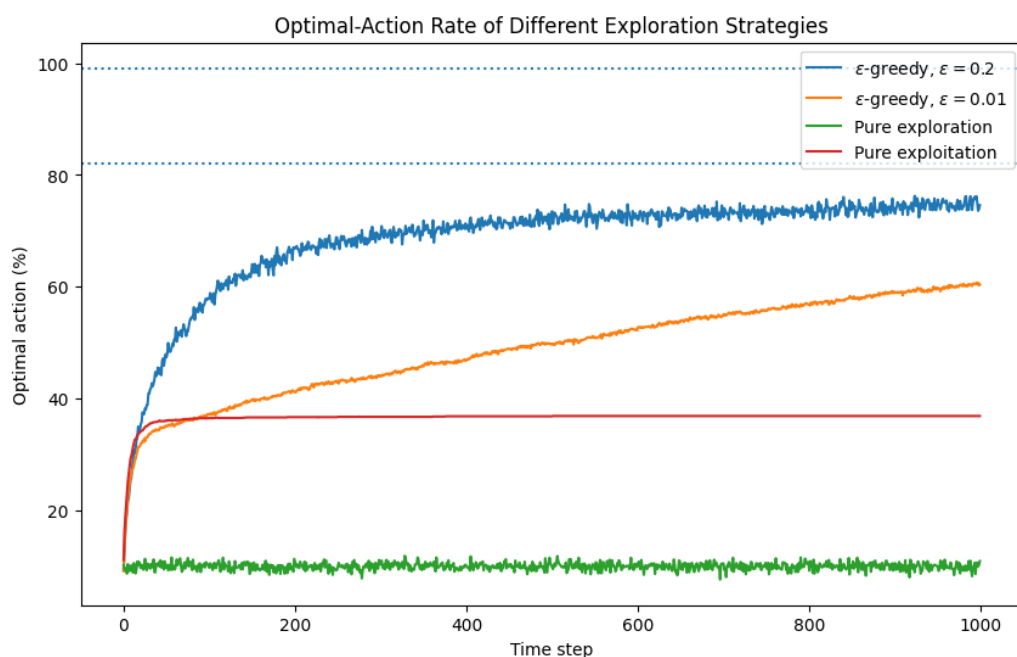
The same four strategies can now be examined through the percentage of optimal actions selected at each step. This view makes the main weakness of pure exploitation particularly clear: its performance quickly plateaus well below 100%. In many of the 2000 bandit problems, early random rewards cause the agent to commit to an action that is not actually optimal. Because a purely greedy strategy never deliberately explores alternatives, it may remain locked onto this suboptimal action indefinitely, with no opportunity to discover its mistake.

```
# Plot the percentage of runs selecting the optimal action at each step
plt.figure(figsize=(10, 6))

plt.plot(res_epsilon_02[1] * 100, label=r"$\varepsilon$-greedy, $\varepsilon=0.2$")
plt.plot(res_epsilon_001[1] * 100, label=r"$\varepsilon$-greedy, $\varepsilon=0.01$")
plt.plot(res_exploration[1] * 100, label="Pure exploration")
plt.plot(res_exploitation[1] * 100, label="Pure exploitation")
```

```
# Show the theoretical maximum optimal-action rate for each epsilon-greedy strategy
plt.axhline((1 - 0.2 + 0.2 / envs[0].k) * 100, linestyle=":")
plt.axhline((1 - 0.01 + 0.01 / envs[0].k) * 100, linestyle=":")

plt.xlabel("Time step")
plt.ylabel("Optimal action (%)")
plt.title("Optimal-Action Rate of Different Exploration Strategies")
plt.legend()
plt.show()
```



The pure exploitation strategy improves **slightly faster** at the beginning because every decision is devoted to exploiting the action that currently appears best. However, its performance soon **plateaus below the optimum**. Early noisy rewards can make a suboptimal action appear best, and without exploration the agent may commit to that action and never gather the information needed to correct its mistake. The epsilon-greedy strategies perform **better in the long run** because they continue to explore. Even after identifying an apparently good action, they occasionally sample alternatives and therefore retain the possibility of discovering a better one. The value of ϵ determines the balance between these two behaviors. A larger value produces **more frequent exploration**, allowing the agent to discover the optimal action more **quickly**. However, this exploration also imposes a **permanent cost**: even after the agent has correctly identified the optimal action, it continues to select other actions during exploratory steps. Assuming a unique optimal action, the maximum probability of selecting it is:

$$P(A_t = a_{\text{greedy}}) = 1 - \varepsilon + \frac{\varepsilon}{k}$$

A smaller value, instead, **explores less frequently**. Learning is therefore **slower**, but once the optimal action has been identified, the agent spends a larger fraction of its time exploiting it. Its **asymptotic optimal-action rate is consequently higher**.

The dotted lines in the figure represent these **theoretical ceilings**. The strategy with greater exploration approaches its ceiling more rapidly, but that ceiling is lower. The strategy with less exploration approaches its ceiling more slowly, but has the potential to achieve a substantially higher optimal-action rate in the long run. This reveals the fundamental limitation of a fixed ε : a larger value provides faster learning at the cost of persistent exploration, whereas a smaller value provides slower learning but better long-term exploitation. This immediately suggests a natural improvement!

1.3.4 Decaying epsilon-greedy

A fixed ε forces the agent to maintain the same exploration rate throughout learning, even though its need for exploration changes over time. Early in learning, the action-value estimates are based on very little experience and are therefore highly uncertain. Exploration is particularly valuable at this stage. As more rewards are observed and the estimates become more reliable, the agent can increasingly exploit what it has learned. This suggests a natural extension of epsilon-greedy: **start with a relatively large value and gradually decrease it over time**. The agent initially explores frequently, then progressively shifts toward exploitation as experience accumulates. This approach is known as **decaying epsilon-greedy**. There are many possible **decay schedules**. For example, the value can decrease linearly, exponentially, or according to some other function of time. Here we use an **exponential decay**, specified by an initial exploration rate, a minimum exploration rate, and the number of steps over which the decay occurs. Rather than recomputing ε at every interaction, we can generate the complete schedule once and simply access the appropriate value during the simulation:

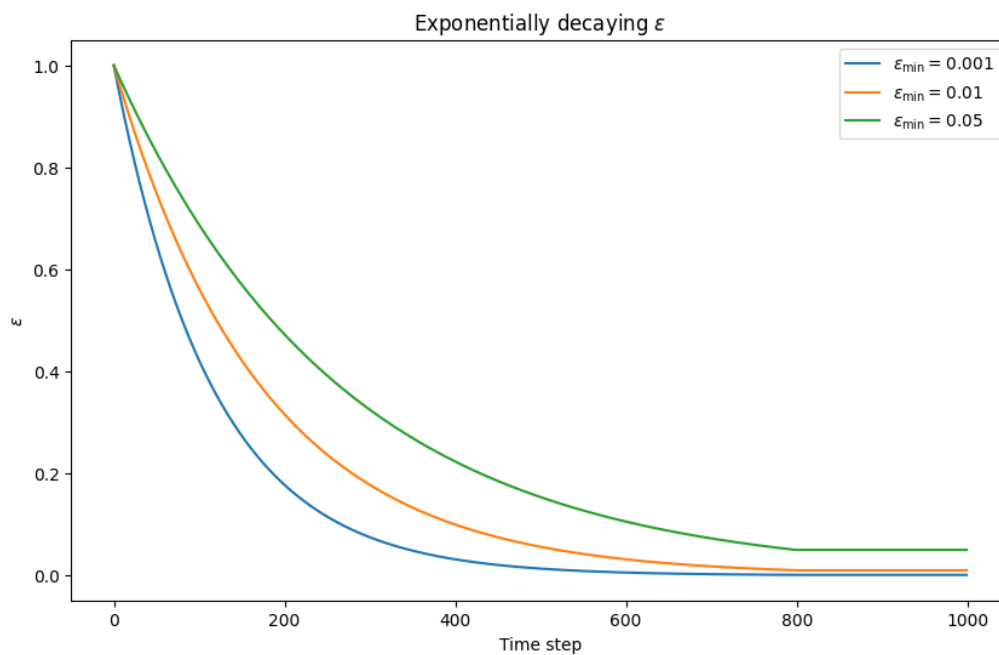
```
def decay_epsilon(init_value, min_value, decay_episodes, max_episodes):  
  
    # Generate an exponential decay from init_value to min_value  
    values = np.geomspace(init_value, min_value, num=decay_episodes)  
  
    # Keep epsilon at its minimum value for the rest of the run  
    values = np.pad(values, (0, max_episodes - decay_episodes), mode="edge")  
  
    return values
```

The resulting schedules show how the minimum value of ϵ determines the amount of persistent exploration that remains after the initial exploration phase:

```
# Plot different epsilon-decay schedules
plt.figure(figsize=(10, 6))

plt.plot(decay_epsilon(1.0, 0.001, 800, 1000), label=r"$\varepsilon_{\min}=0.001$")
plt.plot(decay_epsilon(1.0, 0.01, 800, 1000), label=r"$\varepsilon_{\min}=0.01$")
plt.plot(decay_epsilon(1.0, 0.05, 800, 1000), label=r"$\varepsilon_{\min}=0.05$")

plt.xlabel("Time step")
plt.ylabel(r"$\varepsilon$")
plt.title(r"Exponentially decaying $\varepsilon$")
plt.legend()
plt.show()
```



We can now use this schedule inside the epsilon-greedy strategy. As in the previous experiments, action values are estimated using the sample-average method, only the action-selection rule changes over time:

```
def decay_epsilon_greedy(env, init_epsilon=1.0, min_epsilon=0.001,
    ↪ decay_episodes=100, n_episodes=1000):

    # Initialize the action-value estimates and action-selection counts
```

```
Q = np.zeros(env.k, dtype=float)
N = np.zeros(env.k, dtype=int)

# Store data for later analysis
Q_history = np.empty((n_episodes, env.k), dtype=float)
rewards = np.empty(n_episodes, dtype=float)
actions = np.empty(n_episodes, dtype=int)

# Pre-compute the complete epsilon schedule
epsilons = decay_epsilon(init_epsilon, min_epsilon, decay_episodes, n_episodes)

# Interact with the environment
for e in range(n_episodes):

    # Explore with the current probability epsilon
    if np.random.rand() < epsilons[e]:

        # Select an action uniformly at random
        action = np.random.randint(env.k)

    else:

        # Exploit by selecting uniformly among the greedy actions
        action = np.random.choice(np.flatnonzero(Q == Q.max()))

    # Execute the selected action and observe the reward
    reward = env.step(action)

    # Update the number of times the selected action has been chosen
    N[action] += 1

    # Update its estimated value using the sample-average method
    Q[action] = Q[action] + (reward - Q[action]) / N[action]

    # Store data for later analysis
    Q_history[e] = Q
    rewards[e] = reward
    actions[e] = action

return rewards, Q_history, actions
```

We can evaluate the decaying strategy on the same collection of bandit problems used for the

fixed-epsilon strategies:

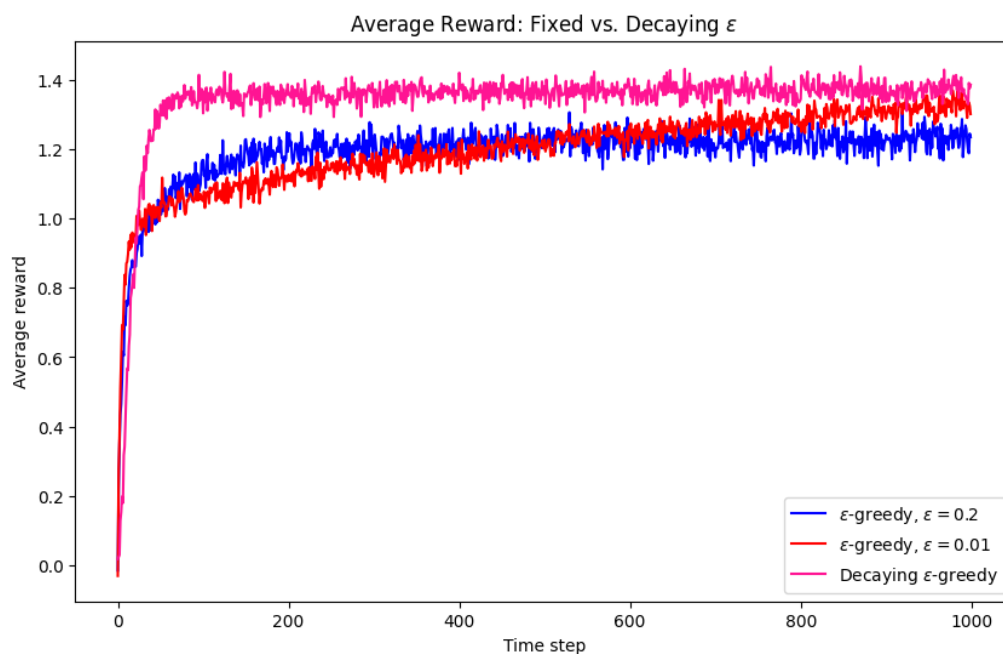
```
# Evaluate epsilon-greedy with decaying epsilon
res_epsilon_decay = evaluate(decay_epsilon_greedy, envs)
```

and compare their average rewards:

```
# Compare fixed and decaying epsilon-greedy strategies
plt.figure(figsize=(10, 6))

plt.plot(res_epsilon_02[0], color="blue", label=r"$\varepsilon$-greedy,
↪ $\varepsilon=0.2$")
plt.plot(res_epsilon_001[0], color="red", label=r"$\varepsilon$-greedy,
↪ $\varepsilon=0.01$")
plt.plot(res_epsilon_decay[0], color="deeppink", label=r"Decaying
↪ $\varepsilon$-greedy")

plt.xlabel("Time step")
plt.ylabel("Average reward")
plt.title(r"Average Reward: Fixed vs. Decaying $\varepsilon$")
plt.legend()
plt.show()
```



The motivation for the decaying strategy is now clear. A relatively large ε at the beginning encourages the agent to sample many different actions while its estimates are still unreliable. As

ϵ decreases, the agent progressively exploits the information acquired during this exploratory phase. Unlike a fixed large ϵ , it does not have to pay the same exploration cost forever; unlike a fixed small ϵ , it does not have to wait a long time to gather useful information about alternative actions. The decay schedule therefore introduces a **new trade-off**. If ϵ decreases **too quickly**, the agent may stop exploring before it has reliably identified the best action. If it decreases **too slowly**, the agent continues wasting interactions on exploration even after its estimates have become sufficiently accurate. The minimum value $\epsilon_{\min}$ determines whether some exploration is retained indefinitely. The central intuition is simple: **explore more when knowledge is scarce**, and **exploit more as knowledge accumulates**. However, this principle relies on the environment being sufficiently stationary. If action values can change over time, reducing exploration too aggressively may prevent the agent from discovering that an action that was previously inferior has become the best one.

1.3.5 Optimistic Initial Values

So far, we have encouraged exploration by modifying the action-selection strategy. For example, epsilon-greedy explicitly forces the agent to select random actions from time to time. However, there is another simple way to induce exploration without changing the greedy action-selection rule at all: we can manipulate the initial action-value estimates. Suppose that all action values are initialized to an **optimistically high value**, larger than the rewards we realistically expect to obtain. The agent then acts greedily:

$$A_t \in \arg \max_a Q_t(a)$$

Initially, all actions look equally promising. When one of them is selected, the observed reward will typically be lower than its optimistic estimate. Its estimate therefore decreases, while the actions that have not yet been tried retain their high initial values and become more attractive. The agent is, in a sense, **repeatedly disappointed** by the actions it tries. Each disappointment encourages it to try another action that still looks promising. As a result, the agent naturally explores different actions even though its action-selection strategy remains completely greedy. This technique is known as **optimistic initial values**. Its mechanism is remarkably simple: initialize the action values optimistically and then act greedily. With the sample-average estimator, the implementation requires only changing the initial values of Q :

```
def optimistic_initialization(env, optimistic_estimate=5.0, n_episodes=1000):  
    # Initialize all action-value estimates optimistically  
    Q = np.full(env.k, optimistic_estimate, dtype=float)
```

```

# Initialize the action-selection counts
N = np.zeros(env.k, dtype=int)

# Store data for later analysis
Q_history = np.empty((n_episodes, env.k), dtype=float)
rewards = np.empty(n_episodes, dtype=float)
actions = np.empty(n_episodes, dtype=int)

# Interact with the environment
for e in range(n_episodes):

    # Act greedily, breaking ties uniformly at random
    action = np.random.choice(np.flatnonzero(Q == Q.max()))

    # Execute the selected action and observe the reward
    reward = env.step(action)

    # Update the action count
    N[action] += 1

    # Update its estimated value using the sample-average method
    Q[action] = Q[action] + (reward - Q[action]) / N[action]

    # Store data for later analysis
    Q_history[e] = Q
    rewards[e] = reward
    actions[e] = action

return rewards, Q_history, actions

```

Notice what happens after an action is selected for the first time. Since $N(a) = 1$, the sample-average update becomes:

$$Q(a) \leftarrow Q_0 + (R - Q_0) = R$$

Thus, the optimistic initialization of that action is completely replaced by its first observed reward. The other actions, however, still retain their optimistic values. They therefore tend to look better and are selected in subsequent steps. This mechanism naturally encourages the agent to try actions that have not yet been sampled. We can compare this strategy with decaying epsilon-greedy:

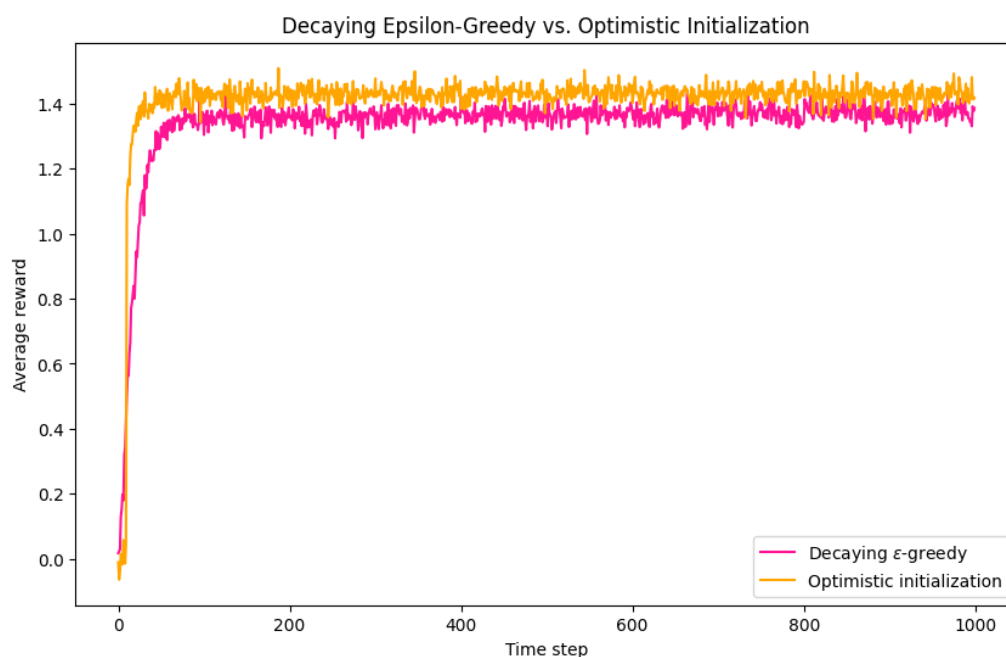
```
# Evaluate the optimistic initialization strategy
res_optimistic = evaluate(optimistic_initialization, envs)

# Compare optimistic initialization with decaying epsilon-greedy

plt.figure(figsize=(10, 6))

plt.plot(res_epsilon_decay[0], color="deeppink", label=r"Decaying  
↪  $\epsilon$ -greedy")
plt.plot(res_optimistic[0], color="orange", label="Optimistic initialization")

plt.xlabel("Time step")
plt.ylabel("Average reward")
plt.title("Decaying Epsilon-Greedy vs. Optimistic Initialization")
plt.legend()
plt.show()
```



At the beginning, optimistic initialization may perform poorly because the agent is effectively driven to try different actions. Once an action has been sampled, its optimistic estimate is replaced by a more realistic value, while unexplored actions continue to look attractive. The agent therefore tends to move through the action set before eventually concentrating on the actions that actually produce larger rewards. An important feature of this method is that exploration is driven by optimism rather than randomness. The agent does not occasionally explore because a

random coin flip tells it to do so, it explores because actions about which it has little experience still appear potentially valuable. However, this also reveals the **main limitation** of optimistic initial values: **the incentive to explore is temporary**. Once every action has been sampled and its optimistic estimate has been corrected by experience, the agent becomes essentially greedy. If the environment later changes, nothing restores the original optimism and therefore nothing creates a renewed incentive to explore. Optimistic initial values can therefore be an effective and extremely simple mechanism for encouraging initial exploration in stationary problems, but they are poorly suited to situations in which continued exploration is required. This limitation motivates exploration strategies that explicitly account for how much is known or remains uncertain about each action.

1.4 Strategic Exploration

Humans explore, but we rarely explore purely at random. Randomness may arise from imperfect knowledge or unpredictable outcomes, but exploration itself is often **strategic**: we deliberately seek information that we expect could improve future decisions. When we explore, we accept a possible short-term cost in exchange for information with long-term value. Moreover, **not every unknown option is equally worth investigating**. We are particularly interested in alternatives that we have **not explored sufficiently** and that **still have the potential** to be better than what we currently know. Consider the restaurant example. We might prefer trying a promising dish that we have never ordered over choosing an unfamiliar dish completely at random. Similarly, there is little reason to keep investigating a dish that we have ordered many times and consistently found mediocre. A sensible exploration strategy should therefore consider not only the estimated value of an action, but also **how uncertain we are about that estimate**. This suggests a more general principle: **exploration should be directed toward actions for which acquiring additional information is potentially valuable**. An action may deserve exploration because it has been sampled only a few times, because its value estimate is highly uncertain, or because it appears sufficiently promising that resolving this uncertainty could change which action we believe to be best. Exploration can also be driven by **surprise** or **prediction error**. If an outcome differs substantially from what we expected, that discrepancy tells us that our current knowledge may be incomplete. Such **unexpected observations** can therefore provide a reason to investigate the corresponding action or situation further.

The strategies considered so far do not explicitly reason about this information. Epsilon-greedy exploration simply chooses a random action with certain probability, treating all exploratory actions equally regardless of their estimated values or uncertainty. More strategic approaches instead use the information already collected to decide where exploration is most useful. They may favor actions according to their estimated value, their uncertainty, their confidence bounds,

or other measures of information gain. Despite its simplicity, however epsilon-greedy remains an **important baseline** because it captures the essential exploration–exploitation trade-off with very little machinery. The methods that follow ask a more sophisticated question: rather than simply deciding whether to explore, can the agent also decide what is most useful to explore?

1.4.1 Softmax method

An informed approach should **explore selectively** rather than blindly. If two actions are still uncertain, but one currently appears much more promising than the other, it makes sense to spend more of our exploration effort on the promising one. At the same time, we should not completely discard actions with lower estimates, because our estimates may still be wrong. We can therefore make every action selectable, but **make better-looking actions more likely to be selected**. The **Softmax strategy** implements this idea by converting the action-value estimates into a probability distribution over actions:

$$\Pr(A_t = a) = \frac{e^{Q_t(a)/\tau}}{\sum_{b=1}^k e^{Q_t(b)/\tau}}$$

Why not simply use the estimated values themselves as probabilities? Action-value estimates are not probabilities: they may be negative, they do not necessarily sum to one, and their absolute scale is arbitrary. Softmax solves this by first transforming each estimate into a positive preference:

$$e^{Q_t(a)/\tau}$$

The exponential function preserves the ordering of the actions, but amplifies differences between their values. If one action has a larger estimated value than another, it receives a larger preference; the greater the difference, the more strongly the first action is favored. The denominator then simply normalizes these preferences so that they sum to one:

$$\sum_{a=1}^k \Pr(A_t = a) = 1$$

As a result, **every action remains possible** because an exponential is always positive, but actions that currently look better are selected more frequently. An especially useful way to understand Softmax is to compare two actions a and b . Their probability ratio is:

$$\frac{\Pr(A_t = a)}{\Pr(A_t = b)} = e^{(Q_t(a) - Q_t(b))/\tau}$$

This reveals that Softmax does not really care about the absolute values of the estimates, but about their differences. If two actions have the same estimated value, they receive the same

probability. If one looks better, its probability grows exponentially relative to the other. The hyperparameter $\tau > 0$, called the **temperature**, controls how sensitive the strategy is to differences between the estimates. A high temperature makes those differences less important, producing a flatter distribution and therefore more exploration. A low temperature amplifies them, concentrating probability on the actions with the largest estimates. The two limiting cases make this interpretation particularly clear:

$$\tau \rightarrow \infty \implies \Pr(A_t = a) \rightarrow \frac{1}{k}$$

so action selection approaches uniform random exploration, whereas:

$$\tau \rightarrow 0^+ \implies \Pr(A_t = a) \rightarrow 0 \text{ for non-greedy actions}$$

and all probability mass concentrates on the greedy action, assuming it is unique. Thus, temperature plays a role similar to ε , but instead of switching between greedy and uniform-random selection, it **continuously controls how strongly action selection favors larger estimated values**. The Softmax probabilities can be computed as follows:

```
def softmax_function(Q, temperature=10.0):

    # Scale the action values by the temperature
    # subtract the maximum for numerical stability
    scaled_Q = (Q - np.max(Q)) / temperature

    # Convert the scaled action values into positive preferences
    exp_Q = np.exp(scaled_Q)

    # Normalize the preferences to obtain a probability distribution
    return exp_Q / exp_Q.sum()
```

We can visualize the role of temperature by keeping the action-value estimates fixed and changing only τ :

```
# Define three fixed action-value estimates
Q = np.array([1.0, 2.0, 3.0])

# Compute the Softmax probabilities for different temperatures
P_01 = softmax_function(Q, temperature=0.1)
P_1 = softmax_function(Q, temperature=1.0)
P_10 = softmax_function(Q, temperature=10.0)

# Define the action positions
```

```

actions = np.arange(len(Q))
width = 0.25

# Plot the selection probabilities
plt.figure(figsize=(8, 5))

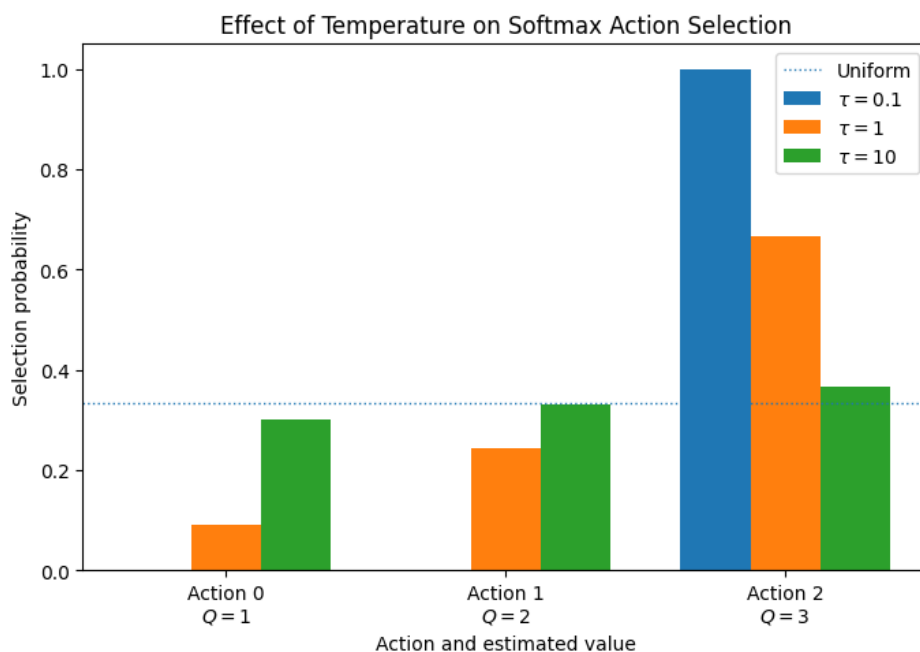
plt.bar(actions - width, P_01, width, label=r"$\tau=0.1$")
plt.bar(actions, P_1, width, label=r"$\tau=1$")
plt.bar(actions + width, P_10, width, label=r"$\tau=10$")

# Show the probability corresponding to uniform random selection
plt.axhline(1 / len(Q), linestyle=":", linewidth=1, label="Uniform")

# Show both the action and its estimated value
plt.xticks(actions, [fr"Action {a}" + "\n" + fr"$Q={Q[a]:g}$" for a in actions])

plt.xlabel("Action and estimated value")
plt.ylabel("Selection probability")
plt.title("Effect of Temperature on Softmax Action Selection")
plt.legend()
plt.show()

```



At low temperature, the action with the largest estimated value receives almost all the probability mass: Softmax behaves nearly like greedy selection. At high temperature, differences

between action values are compressed and the probabilities approach the uniform distribution. Intermediate temperatures provide a graded form of exploration, favoring promising actions without completely excluding the others.

As with ϵ , the **temperature does not need to remain constant**. Early in learning, when action-value estimates are uncertain, we may want a high temperature and broad exploration. Later, as the estimates become more reliable, we can progressively reduce the temperature and concentrate probability on the actions that appear best. For example, we can define a linearly decreasing temperature schedule:

```
def decay_temperature(init_temp, min_temp, decay_episodes, max_episodes):  
  
    # Linearly decrease the temperature  
    values = np.linspace(init_temp, min_temp, num=decay_episodes)  
  
    # Keep the minimum temperature for the rest of the run  
    values = np.pad(values, (0, max_episodes - decay_episodes), mode="edge")  
  
    return values
```

The Softmax strategy then has essentially the same structure as the previous methods. The action-value estimator remains unchanged; only the action-selection mechanism is different:

```
def softmax(env, init_temp=1.0, min_temp=0.1, decay_ratio=0.5, n_episodes=1000):  
  
    # Initialize the action-value estimates and action-selection counts  
    Q = np.zeros(env.k, dtype=float)  
    N = np.zeros(env.k, dtype=int)  
  
    # Store data for later analysis  
    Q_history = np.empty((n_episodes, env.k), dtype=float)  
    rewards = np.empty(n_episodes, dtype=float)  
    actions = np.empty(n_episodes, dtype=int)  
  
    # Determine how many episodes are used for temperature decay  
    decay_episodes = max(1, int(n_episodes * decay_ratio))  
  
    # Pre-compute the complete temperature schedule  
    temperatures = decay_temperature(init_temp, min_temp, decay_episodes, n_episodes)  
  
    # Interact with the environment  
    for e in range(n_episodes):
```

```
# Convert the current action-value estimates into probabilities
probabilities = softmax_function(Q, temperature=temperatures[e])

# Sample an action according to the Softmax probabilities
action = np.random.choice(env.k, p=probabilities)

# Execute the selected action and observe the reward
reward = env.step(action)

# Update the action count
N[action] += 1

# Update its estimated value using the sample-average method
Q[action] = Q[action] + (reward - Q[action]) / N[action]

# Store data for later analysis
Q_history[e] = Q
rewards[e] = reward
actions[e] = action

return rewards, Q_history, actions
```

The key difference from epsilon-greedy is now explicit. During an exploratory epsilon-greedy step, all actions are treated equally. With Softmax, exploration is biased toward actions that already appear promising. An action estimated to be almost as good as the current best action receives substantially more attention than one whose estimated value is very poor. This makes Softmax exploration more selective, but it introduces an **important limitation**: the probabilities depend on the relative scale of the action-value estimates. The same temperature can produce very different behavior when the differences between estimated values are small or large. Consequently, choosing and scheduling τ can require **careful tuning**.

1.4.2 Upper Confidence Bound method

Optimistic initialization encourages exploration by assigning artificially high values to actions at the beginning. Its main limitation is that this optimism is **arbitrary and temporary**: we must decide how optimistic to be, and once the initial estimates have been corrected, the incentive to explore disappears. A more principled approach is to make optimism depend on **how uncertain we are about each action**. If an action has been sampled many times, we can be relatively confident in its estimated value. If it has been sampled only a few times, its true value could

still be substantially better than its current estimate suggests. This is the idea behind the **Upper Confidence Bound (UCB)** method. Instead of selecting actions based only on their estimated value, UCB adds an **uncertainty bonus**:

$$A_t = \arg \max_a [Q_t(a) + c u_t(a)]$$

An action can therefore be attractive for two reasons: **because we already believe it is good**, or **because we are still uncertain about how good it might be**. UCB effectively gives uncertain actions the **benefit of the doubt**. The uncertainty bonus should behave naturally: it should decrease when an action is sampled, because we learn more about it, and increase when an action is neglected, giving it a chance to be reconsidered. A standard choice satisfying these properties is:

$$u_t(a) = \sqrt{\frac{\ln t}{N_t(a)}}$$

As the number of times an action is selected increases, the bonus decreases because the action is becoming better known. If the action is not selected, the denominator remains fixed while t increases, so its bonus slowly grows and the action eventually becomes worth reconsidering. Thus, **sampling reduces uncertainty**, while **neglecting gradually restores the incentive to explore**. This gives the UCB selection rule:

$$A_t = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$

Promising actions receive more attention, but even neglected actions are occasionally reconsidered as their relative uncertainty increases. Thus, **exploration emerges from the agent's uncertainty about its knowledge**, rather than from random action selection. The parameter c determines **how much importance is assigned to uncertainty**. A small c favors exploitation, while a larger c encourages more exploration. Unlike epsilon-greedy, which explores by selecting actions uniformly at random, UCB performs directed exploration: it favors actions that are either promising or insufficiently explored. Before applying the formula, notice that every action must be sampled at least once because the uncertainty term is undefined when the denominator is 0. We therefore begin by selecting each action once and then apply the UCB rule:

```
def upper_confidence_bound(env, c=2.0, n_episodes=1000):

    # Initialize the action-value estimates and action-selection counts
    Q = np.zeros(env.k, dtype=float)
    N = np.zeros(env.k, dtype=int)

    # Store data for later analysis
    Q_history = np.empty((n_episodes, env.k), dtype=float)
```

```
rewards = np.empty(n_episodes, dtype=float)
actions = np.empty(n_episodes, dtype=int)

# Interact with the environment
for e in range(n_episodes):

    # Select every action once before applying the UCB rule
    if e < env.k:
        action = e

    else:
        # Current time step
        t = e + 1

        # Compute the uncertainty bonus for each action
        uncertainty = np.sqrt(np.log(t) / N)

        # Combine estimated value and uncertainty
        UCB = Q + c * uncertainty

        # Select uniformly at random among actions with the highest UCB score
        action = np.random.choice(np.flatnonzero(UCB == UCB.max()))

    # Execute the selected action and observe the reward
    reward = env.step(action)

    # Update the action count
    N[action] += 1

    # Update its estimated value using the sample-average method
    Q[action] = Q[action] + (reward - Q[action]) / N[action]

    # Store data for later analysis
    Q_history[e] = Q
    rewards[e] = reward
    actions[e] = action

return rewards, Q_history, actions
```

UCB can perform very well on stationary multi-armed bandits. Its main limitation is that the uncertainty mechanism relies on counts and assumptions that become harder to exploit directly in more general reinforcement learning problems, particularly with non-stationarity, large state

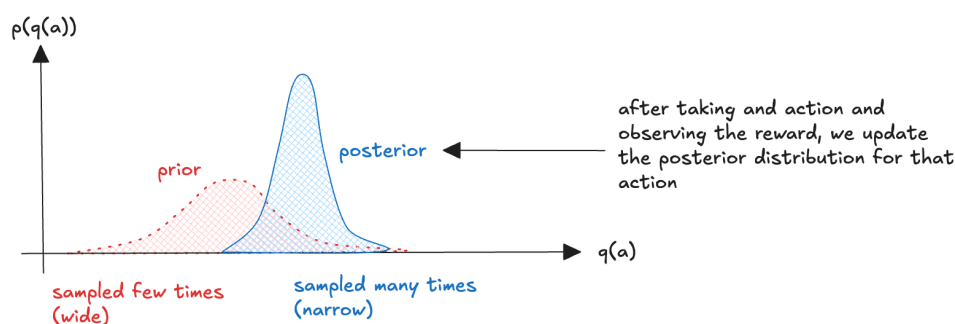
spaces, and function approximation.

1.4.3 Thompson sampling

UCB uses uncertainty explicitly, but compresses it into a single exploration bonus. **Thompson sampling** takes a different approach: instead of summarizing uncertainty with one number, it maintains an entire **probability distribution over the possible value of each action**. This introduces an important distinction from the methods considered so far. Recall that $q(a)$ denotes the true but unknown action value, whereas $Q_t(a)$ is only our current point estimate of that value. Instead of representing our knowledge with a single estimate, Thompson sampling represents all the values that the unknown $q(a)$ could plausibly take, together with how likely we currently believe each of them to be. For each action, the agent starts with a **prior distribution**, representing what it believes about the unknown action value before observing any rewards.



As rewards are collected, the prior is updated into a **posterior distribution**, which incorporates the evidence observed so far. An action that has been sampled many times will typically have a narrow posterior: we have a fairly precise idea of its true value. An action that has been sampled only a few times will have a wider posterior: many different values are still plausible.



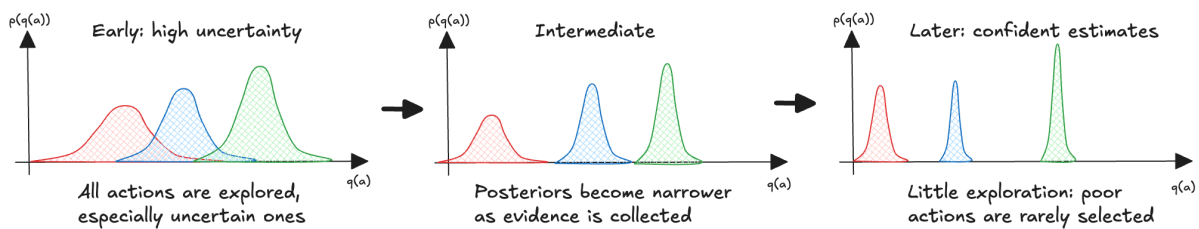
Thompson sampling turns these distributions directly into decisions. At every step, it draws one plausible value of the unknown $q(a)$ from the posterior of each action:

$$\tilde{q}_t(a) \sim p(q(a) \mid \mathcal{D}_t)$$

where $\mathcal{D}_t$ represents all the evidence collected up to time t . Notice that $\tilde{q}_t(a)$ is neither the true value nor a point estimate: it is one random hypothesis about what the true value might be, given the evidence available so far. The agent then acts greedily with respect to these sampled hypotheses:

$$A_t = \arg \max_a \tilde{q}_t(a)$$

Thus, Thompson sampling replaces the single question "What is my best estimate of each action?" with "Given what I currently know, what could each action's true value be?" The randomness in the answer naturally generates exploration. The intuition is particularly elegant. An action with a high estimated value often produces a large sample and is therefore frequently selected. But **an uncertain action can occasionally produce a very optimistic sample** and win the comparison even if its posterior mean is lower. **Uncertainty itself creates exploration**. As more evidence is collected about an action, **its posterior becomes narrower**. If the action is actually poor, optimistic samples become increasingly unlikely and the agent naturally stops exploring it. If it is good, its posterior concentrates around a high value and it continues to be selected. There is therefore no explicit distinction between an exploration step and an exploitation step. Both emerge from the same sampling mechanism.



To implement this idea, **we need to choose a probability distribution** representing our **initial belief** about each unknown action value. One convenient choice is a Gaussian prior:

$$q(a) \sim \mathcal{N}(Q_0, \sigma_0^2)$$

Here, the prior mean represents our initial guess of the true action value, while the prior variance represents how uncertain we are about that guess. A small variance means that we strongly believe the true value is close to the mean; a large variance means that a much wider range of values is considered plausible. As rewards are observed, we use them as evidence to refine this initial belief. To do so, we also need an **observation model** describing how rewards are generated for a given true action value. We assume that rewards are noisy observations centered around the true value:

$$R \mid q(a) \sim \mathcal{N}(q(a), \sigma^2)$$

The same true value appears in both distributions and always has the same meaning: it is the true but unknown action value. The two variances, instead, describe two different sources of

uncertainty. The prior variance describes our uncertainty about where the true action value is, whereas the reward variance describes how much individual rewards fluctuate around that true value. This distinction is important. A reward far from our current belief is strong evidence if rewards are normally very stable, but much weaker evidence if rewards are intrinsically very noisy. The observation model therefore determines how much information each new reward provides about the unknown action value. A useful property of this model is that combining a Gaussian prior with Gaussian observations produces another Gaussian distribution. After observing the available evidence, our belief about the action value becomes the posterior:

$$q(a) \mid \mathcal{D}_t \sim \mathcal{N}(\mu_t(a), \sigma_t^2(a))$$

As rewards are collected, we need to **update two aspects of our belief**: its **center**, which tells us where we think the true action value is, and its **uncertainty**, which tells us how confident we are in that belief. We have two sources of information about the unknown action value: the prior mean, representing our initial belief, and the sample mean, representing what we have learned from observed rewards. These two quantities should not contribute equally. The one supported by more precise information should receive more weight. For Gaussian distributions, it is convenient to express confidence through precision, defined as the inverse of variance:

$$\lambda_0 = \frac{1}{\sigma_0^2} \quad \lambda_R = \frac{1}{\sigma^2}$$

After action a has been selected $N_t(a)$ times, the observed data contribute a total precision equal to:

$$N_t(a)\lambda_R$$

The **posterior mean** is therefore a precision-weighted average of the prior mean and the sample mean:

$$\mu_t(a) = \frac{\lambda_0 Q_0 + N_t(a)\lambda_R Q_t(a)}{\lambda_0 + N_t(a)\lambda_R}$$

This equation has a simple interpretation. When only a few rewards have been observed, the prior still has substantial influence. As more rewards are collected, the weight of the data increases and the posterior mean moves progressively toward the sample mean. Once the center has been updated, we can update the uncertainty. Precisions add as independent evidence accumulates, so the **posterior precision** is:

$$\lambda_t(a) = \lambda_0 + N_t(a)\lambda_R$$

and therefore the posterior variance is:

$$\sigma_t^2(a) = \frac{1}{\lambda_t(a)}$$

This captures the second effect of experience: the **more observations we collect, the more precise our belief becomes**. As $N_t(a)$ increases, the posterior precision increases and the posterior variance decreases. The implementation follows the mathematics directly:

```
def thompson_sampling(env, prior_mean=0.0, prior_sigma=5.0, reward_sigma=1.0,
    ↪ n_episodes=1000):

    # Initialize the sample means and action-selection counts
    Q = np.zeros(env.k, dtype=float)
    N = np.zeros(env.k, dtype=int)

    # Store data for later analysis
    Q_history = np.empty((n_episodes, env.k), dtype=float)
    rewards = np.empty(n_episodes, dtype=float)
    actions = np.empty(n_episodes, dtype=int)

    # Interact with the environment
    for e in range(n_episodes):

        # Compute the posterior variance of each action
        posterior_var = 1.0 / (1.0 / prior_sigma**2 + N / reward_sigma**2)

        # Compute the posterior mean of each action
        posterior_mean = posterior_var * (prior_mean / prior_sigma**2 + N * Q /
    ↪ reward_sigma**2)

        # Sample one plausible action value from each posterior
        sampled_values = np.random.normal(posterior_mean, np.sqrt(posterior_var))

        # Act greedily with respect to the sampled values
        action = np.random.choice(np.flatnonzero(sampled_values ==
    ↪ sampled_values.max()))

        # Execute the selected action and observe the reward
        reward = env.step(action)

        # Update the action count
        N[action] += 1

        # Update its sample-average estimate
        Q[action] = Q[action] + (reward - Q[action]) / N[action]

        # Store data for later analysis
```

```
Q_history[e] = Q
rewards[e] = reward
actions[e] = action

return rewards, Q_history, actions
```

A contrast with UCB is useful. Both methods use uncertainty to guide exploration, but they use it differently. UCB is **deterministically optimistic**: it evaluates each action near the optimistic side of what is plausible. Thompson sampling is **probabilistically optimistic**: it randomly draws one plausible value and acts as though that value were true. As uncertainty decreases, the randomness of those samples decreases automatically. Exploration therefore switches itself off when the evidence becomes convincing, without requiring an explicit parameter schedule. The price for this elegant behavior is that Thompson sampling **requires a probabilistic model** of the unknown action values and rewards: that model is what buys the exploration for free.

1.4.4 Comparison

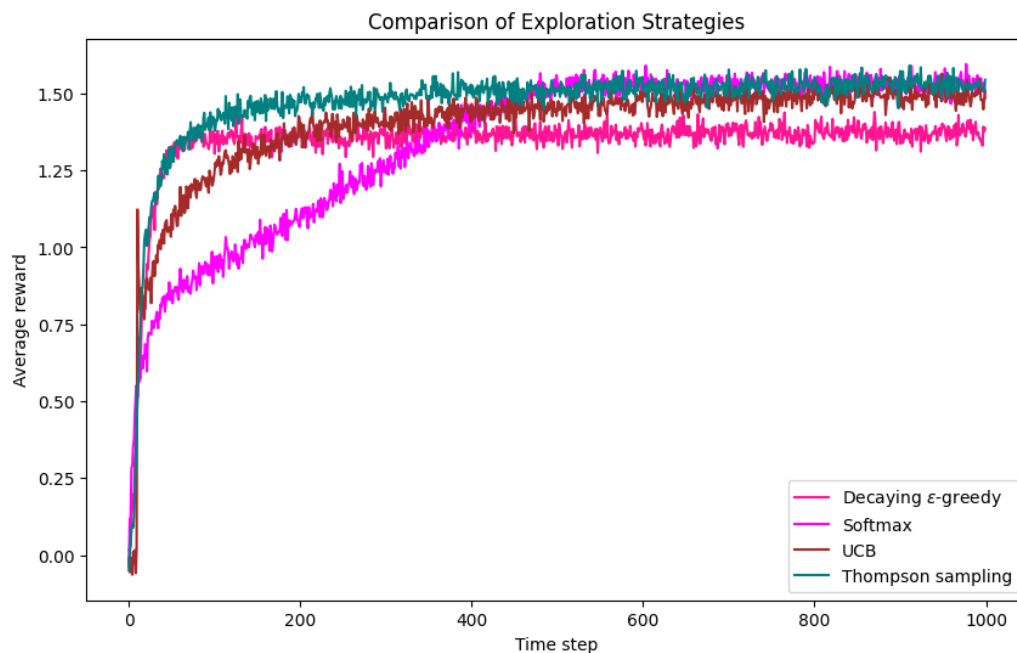
We can now compare the strategic exploration methods introduced in this section with decaying epsilon-greedy. All methods face exactly the same bandit problems, so differences in performance reflect how each strategy decides where and when to explore.

```
# Evaluate the strategic exploration methods
res_softmax = evaluate(softmax, envs)
res_ucb = evaluate(upper_confidence_bound, envs)
res_thompson = evaluate(thompson_sampling, envs)

# Compare the average reward of the different strategies
plt.figure(figsize=(10, 6))

plt.plot(res_epsilon_decay[0], color="deeppink", label=r"Decaying
↳ $\epsilon$-greedy")
plt.plot(res_softmax[0], color="magenta", label="Softmax")
plt.plot(res_ucb[0], color="brown", label="UCB")
plt.plot(res_thompson[0], color="teal", label="Thompson sampling")

plt.xlabel("Time step")
plt.ylabel("Average reward")
plt.title("Comparison of Exploration Strategies")
plt.legend()
plt.show()
```



The four methods differ primarily in how they decide which actions are worth exploring. Decaying epsilon-greedy explores without distinguishing among actions, although it gradually reduces how often it does so. Softmax uses the current value estimates to make promising actions more likely to be explored. UCB goes one step further and explicitly combines estimated value and uncertainty, directing exploration toward actions that are either good or insufficiently known. Thompson sampling represents that uncertainty with an entire posterior distribution and lets random samples from those distributions determine which action to try. In this experiment, the three strategic methods rapidly achieve and maintain high average rewards, illustrating the advantage of directing exploration using the information already collected rather than exploring uniformly at random. **These benefits come with different requirements.** Softmax requires choosing a temperature and its schedule. UCB requires an exploration coefficient c and an explicit measure of uncertainty. Thompson sampling requires a probabilistic model of the unknown quantities and observations, together with a prior. Moreover, the reward curves are relatively close and therefore hide some of the differences in how efficiently the strategies explore.

1.5 Regret: Measuring the Cost of Exploration

So far, we have compared exploration strategies mainly through the reward they collect. Bandit theory often looks at the same problem from the opposite perspective: instead of asking how much reward did the agent obtain? we ask **how much reward did it lose** because it did not always select the optimal action? This quantity is called **regret**, and it provides a natural way to

measure the cost of learning. For every action different from the optimal one, we define its **gap** as:

$$\Delta_a = q(a_*) - q(a) \geq 0$$

The gap measures the **expected cost of selecting action a** once instead of selecting the optimal action. A nearly optimal action has a small gap, whereas a poor action has a large one. Over T interactions, these individual losses accumulate. The **cumulative regret** of a sequence of selected actions is:

$$L_T = \sum_{t=1}^T [q(a_*) - q(A_t)]$$

This expression reveals an important property of regret: exploring a suboptimal action is not equally expensive in every case. Trying an action whose value is close to the optimum costs very little; trying a very poor action costs much more. Taking the expectation over the randomness of the algorithm and rewards gives the **expected cumulative regret**:

$$\mathbb{E}[L_T] = \sum_a \mathbb{E}[N_T(a)] \Delta_a$$

Regret depends both on **how bad a suboptimal action is** and on **how often the agent selects it**. A good exploration strategy should therefore learn to stop spending interactions on actions once there is enough evidence that they are suboptimal. Maximizing expected cumulative reward and minimizing expected cumulative regret are two views of the same objective. Regret, however, gives us a particularly useful reference point: zero regret corresponds to an oracle that already knows the optimal action and selects it every time. What matters most is not simply the amount of regret accumulated after a particular number of steps, but **how regret grows as the horizon becomes longer**. A strategy with linear regret continues paying a non-vanishing cost per interaction:

$$L_T = \Theta(T)$$

Pure random exploration has this behavior because it continues selecting suboptimal actions forever. Pure greedy exploitation can also suffer linear regret if it commits to the wrong action. A fixed-epsilon strategy likewise continues paying for exploration indefinitely because a fixed fraction of its decisions remains exploratory. A successful learning strategy should instead achieve sublinear regret:

$$\frac{L_T}{T} \rightarrow 0$$

This means that the average loss per interaction eventually vanishes: although the agent must pay some cost to learn which actions are good, that cost becomes negligible over a sufficiently long run. For classical stochastic bandits, important algorithms can achieve regret that grows only logarithmically with time:

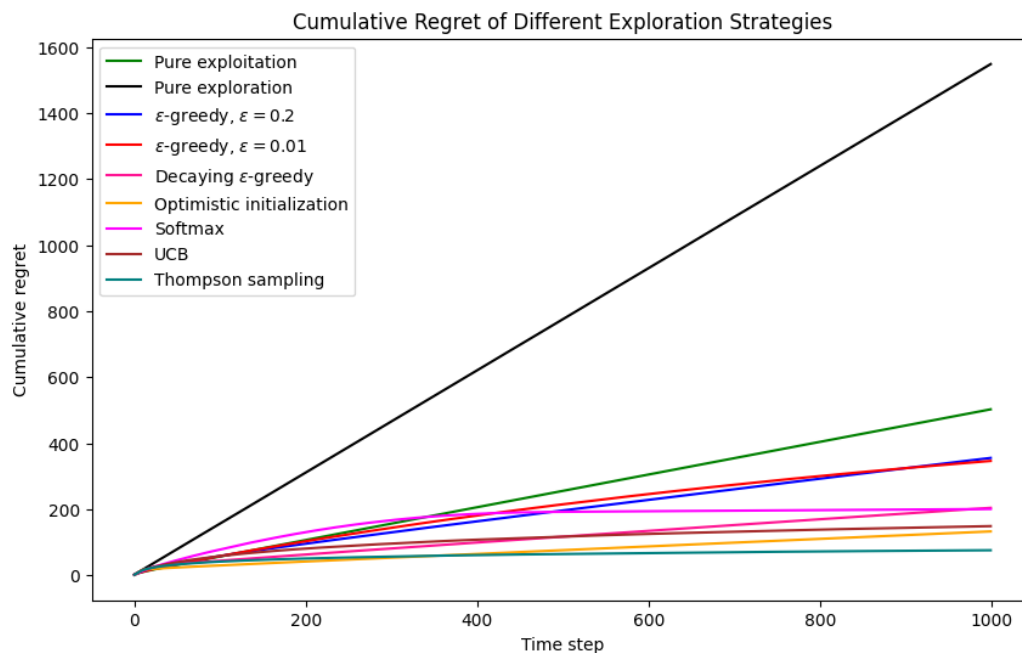
$$L_T = O(\ln T)$$

The logarithmic dependence has a particularly important interpretation: **the agent continues exploring, but increasingly rarely**. It gathers only the additional information necessary to remain confident about which action is best. Classical lower-bound results show that logarithmic problem-dependent regret is the fundamental asymptotic scale for sufficiently well-behaved stochastic bandit problems; appropriately designed UCB and Thompson-sampling methods can attain this scale under their respective assumptions. This gives the regret plot a useful visual interpretation:

```
# Plot the cumulative regret of all strategies
plt.figure(figsize=(10, 6))

plt.plot(res_exploitation[2], color="green", label="Pure exploitation")
plt.plot(res_exploration[2], color="black", label="Pure exploration")
plt.plot(res_epsilon_02[2], color="blue", label=r"$\varepsilon$-greedy,
↪ $\varepsilon=0.2$")
plt.plot(res_epsilon_001[2], color="red", label=r"$\varepsilon$-greedy,
↪ $\varepsilon=0.01$")
plt.plot(res_epsilon_decay[2], color="deeppink", label=r"Decaying
↪ $\varepsilon$-greedy")
plt.plot(res_optimistic[2], color="orange", label="Optimistic initialization")
plt.plot(res_softmax[2], color="magenta", label="Softmax")
plt.plot(res_ucb[2], color="brown", label="UCB")
plt.plot(res_thompson[2], color="teal", label="Thompson sampling")

plt.xlabel("Time step")
plt.ylabel("Cumulative regret")
plt.title("Cumulative Regret of Different Exploration Strategies")
plt.legend()
plt.show()
```



The figure exposes differences that can be difficult to see from average reward alone. **Pure exploration accumulates regret almost linearly** because it continues allocating interactions uniformly across all actions, including poor ones. It learns their values accurately, but never uses that knowledge to stop selecting them. **Pure exploitation can also accumulate approximately linear regret**, but for the opposite reason. It stops exploring too soon and may commit to a suboptimal action. Once that happens, it repeatedly pays the same gap because it never collects the information required to correct its decision. Fixed-epsilon strategies illustrate another important point. Even after the optimal action has been identified, they continue exploring with a fixed probability. Their learning may therefore improve dramatically, but a permanent exploration cost remains, producing **asymptotically linear regret**. Strategies such as UCB and Thompson sampling behave differently. As evidence accumulates, they become increasingly selective about exploration. Poorly performing actions are sampled less and less frequently, so the additional regret accumulated at each new step decreases. The corresponding curves therefore **tend to bend rather than continue along a fixed linear slope**. This is the central message of regret analysis: **exploration is not free**, but good exploration pays a finite and progressively smaller price for acquiring the information needed to make better decisions. We can also compare the final cumulative regret numerically:

```
print(f"{'Strategy':<24}{'Final regret':>16}")

print(f"{'Pure exploitation':<24}{res_exploitation[2][-1]:>16.1f}")
print(f"{'Pure exploration':<24}{res_exploration[2][-1]:>16.1f}")
```

```

print(f"{'Epsilon-greedy 0.2':<24}{res_epsilon_02[2][-1]:>16.1f}")
print(f"{'Epsilon-greedy 0.01':<24}{res_epsilon_001[2][-1]:>16.1f}")
print(f"{'Decaying epsilon':<24}{res_epsilon_decay[2][-1]:>16.1f}")
print(f"{'Optimistic':<24}{res_optimistic[2][-1]:>16.1f}")
print(f"{'Softmax':<24}{res_softmax[2][-1]:>16.1f}")
print(f"{'UCB':<24}{res_ucb[2][-1]:>16.1f}")
print(f"{'Thompson sampling':<24}{res_thompson[2][-1]:>16.1f}")

```

Strategy	Final regret
Pure exploitation	502.5
Pure exploration	1548.2
Epsilon-greedy 0.2	355.2
Epsilon-greedy 0.01	346.1
Decaying epsilon	204.0
Optimistic	132.6
Softmax	200.5
UCB	148.7
Thompson sampling	75.6

1.6 Parameter study

We have now introduced several strategies for balancing exploration and exploitation, but **comparing them using a single parameter setting can be misleading**. Each method behaves differently depending on its **hyperparameters**. For example, ϵ controls the amount of random exploration in epsilon-greedy, c controls the importance of uncertainty in UCB, and the temperature controls how strongly Softmax favors actions with larger estimated values. A fair comparison should therefore ask not only how well each method performs, but also **how sensitive its performance is to the choice of its parameters**. To investigate this, we can perform a **parameter study**. For each strategy, we vary its main parameter over a wide range and summarize the resulting **learning curve** by the average reward obtained over the complete run. Each point in the resulting plot therefore represents the performance of one strategy with one particular parameter setting, averaged over all independent bandit problems. Because the parameters have very different numerical ranges, we place them on a common logarithmic scale. The horizontal position should therefore be interpreted as the value of the parameter appropriate to each method, not as the same quantity shared across algorithms.

```
# Epsilon-greedy: vary epsilon
egreedy_x, egreedy_y = [], []
for epsilon in [1/128, 1/64, 1/32, 1/16, 1/8, 1/4]:
    result = evaluate(epsilon_greedy, envs, epsilon=epsilon, n_episodes=1000)
    egreedy_x.append(epsilon)
    egreedy_y.append(result[0].mean())

# Optimistic initialization: vary the initial action-value estimate
opt_x, opt_y = [], []
for Q0 in [1/4, 1/2, 1, 2, 4, 8]:
    result = evaluate(optimistic_initialization, envs, optimistic_estimate=Q0,
        ↪ n_episodes=1000)
    opt_x.append(Q0)
    opt_y.append(result[0].mean())

# UCB: vary the exploration coefficient
ucb_x, ucb_y = [], []
for c in [1/16, 1/8, 1/4, 1/2, 1, 2, 4]:
    result = evaluate(upper_confidence_bound, envs, c=c, n_episodes=1000)
    ucb_x.append(c)
    ucb_y.append(result[0].mean())

# Softmax: vary the initial temperature
softmax_x, softmax_y = [], []
for temperature in [1/16, 1/8, 1/4, 1/2, 1, 2, 4]:
    result = evaluate(softmax, envs, init_temp=temperature, min_temp=min(0.1,
        ↪ temperature/10), decay_ratio=0.04,
        ↪ n_episodes=1000)
    softmax_x.append(temperature)
    softmax_y.append(result[0].mean())

# Thompson sampling: vary the prior standard deviation
thompson_x, thompson_y = [], []
for prior_sigma in [1/16, 1/8, 1/4, 1/2, 1, 2, 4]:
    result = evaluate(thompson_sampling, envs, prior_sigma=prior_sigma,
        ↪ n_episodes=1000)
    thompson_x.append(prior_sigma)
    thompson_y.append(result[0].mean())
```

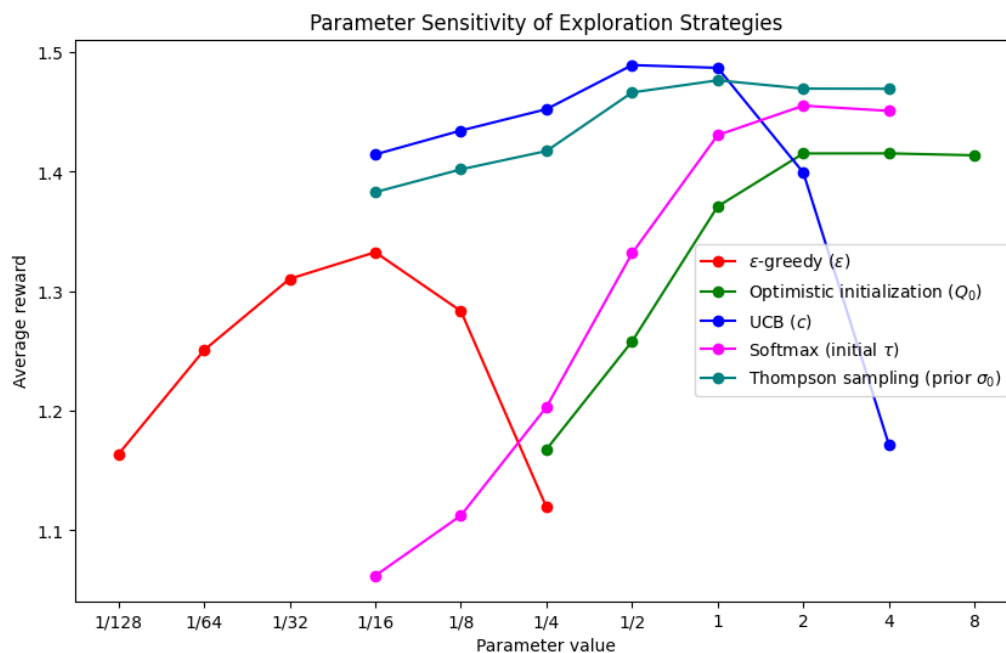
We can now compare the sensitivity of the different strategies:

```
# Compare the sensitivity of each strategy to its main parameter
plt.figure(figsize=(10, 6))

plt.plot(np.log2(egreedy_x), egreedy_y, "o-", color="red",
         label=r"$\varepsilon$-greedy ($\varepsilon$)")
plt.plot(np.log2(opt_x), opt_y, "o-", color="green", label=r"Optimistic
         initialization ($Q_0$)")
plt.plot(np.log2(ucb_x), ucb_y, "o-", color="blue", label=r"UCB ($c$)")
plt.plot(np.log2(softmax_x), softmax_y, "o-", color="magenta", label=r"Softmax
         (initial $\tau$)")
plt.plot(np.log2(thompson_x), thompson_y, "o-", color="teal", label=r"Thompson
         sampling (prior $\sigma_0$)")

# Show the actual parameter values instead of their log2 values
plt.xticks(range(-7, 4), ("1/128", "1/64", "1/32", "1/16", "1/8", "1/4", "1/2", "1",
         "2", "4", "8"))

plt.xlabel("Parameter value")
plt.ylabel("Average reward")
plt.title("Parameter Sensitivity of Exploration Strategies")
plt.legend()
plt.show()
```



The parameter study reveals an important property that cannot be seen from a single learning curve. For strategies with an explicit exploration parameter, **performance typically deteriorates at both extremes**. Too little exploration prevents the agent from collecting enough information to reliably identify the best action, too much exploration prevents it from fully exploiting what it has learned. Good performance therefore tends to occur over an intermediate range of parameter values. The **width of this high-performing region** is itself important. A method that performs well only for a very specific parameter value requires **careful tuning**, whereas a method whose performance remains high across a broad range is **more robust to parameter choice**. The main lesson of the parameter study is therefore not simply which curve reaches the highest point. It is that an exploration strategy should be evaluated along two dimensions: how well it can perform when appropriately configured, and how robust that performance is to the choice of its parameters.

1.7 From Immediate Rewards to Long-Term Value

Everything in this chapter has taken place in the simplest possible decision problem: a world with a single state and immediate consequences. However in general, actions don't determine only the reward received now, they will also influence the states visited next and, through them, the rewards that can be obtained in the **future**. The agent must therefore learn not only which actions are valuable, but which actions are valuable in each state. Remarkably, the fundamental learning mechanism does not change. We will continue to use the same incremental update introduced for sample averages:

$$\text{NewEstimate} \leftarrow \text{OldEstimate} + \text{StepSize} \times (\text{Target} - \text{OldEstimate})$$

What changes is the **target**. In a bandit, the target is simply the reward observed after selecting an action. In a sequential problem, the value of an action depends on **what happens afterward**, so the target must somehow incorporate future rewards. We will see two fundamental ways of doing this: **waiting and accumulating** the rewards that actually follow, or **updating earlier** by combining the immediate reward with the agent's current estimate of what comes next.

This also brings back two ideas developed in this chapter. First, learning becomes inherently dynamic because the targets may themselves depend on estimates that are still changing; step-size choices therefore remain central to controlling how quickly new information replaces old information. Second, the exploration–exploitation dilemma does not disappear when states are introduced, it becomes richer. Instead of choosing among actions according to $Q(a)$, the agent will choose among them according to state-dependent action values $Q(s, a)$.

The bandit problem has therefore given us more than a collection of exploration strategies. It has introduced the basic machinery that we will reuse in sequential reinforcement learning:

estimate values from experience, update them from an error signal, and use those estimates to decide what to do next.